`fall_n` — Technical Documentation

Simplex Elements, Quadrature Theory,
Element Architecture, and Implementation Decisions

S. E. Chavarría-Miranda

March 12, 2026

Abstract

This document provides a comprehensive technical reference for the `fall_n` nonlinear finite element library. It covers the implementation of simplex (tetrahedral) finite elements, numerical integration strategies on simplicial domains, the engineering decisions made to ensure robust post-processing, and the layered element architecture with its policy-based design patterns.

Key topics include:

- Lagrange shape functions on the reference simplex and their isoparametric mapping to physical space;
- Classical simplex quadrature rules and their negative-weight pathology;
- The Stroud conical product rule with Gauss–Jacobi nodes via the Golub–Welsch algorithm as an all-positive-weight alternative;
- Diagnosis and resolution of the checkerboard artefact in lumped L^2 projections on quadratic tetrahedra;
- Volume-weighted (SPR-type) nodal averaging as a robust projection method;
- The refactor of `ElementGeometry`: corrected value semantics, explicit quadrature strategy contracts, compile-time invariant enforcement, and preservation of runtime performance in the hot path;
- Serendipity tensor-product and simplex element families, including static reference cells, benchmark validation, VTK support, and Gmsh import paths for HEX20 and QUA8;
- A detailed ontology for the `Domain` refactor, separating geometry, mesh vertices, nodal unknown carriers, integration sites, constitutive points, and structural section entities under a compile-time-oriented HPC design philosophy;
- The first implementation phase of the `Domain` refactor: canonical `Vertex` storage, lazy PETSc mesh creation, transitional nodal caching, and geometry-only mesh export through vertices;
- The second implementation phase of the `Domain` refactor: geometry-only element binding through `Point/Vertex`, optional analysis-node binding, vertex-driven boundary extraction, and preservation of the compile-time/HPC characteristics of the element kernels;
- A bug fix in PETSc 3.24.5 `section.c` and application-level workarounds for PETSc indexing and preallocation issues;
- The element architecture design: External Polymorphism at the geometry and assembly levels, policy-based customisation points (MaterialPolicy, KinematicPolicy, ElementPolicy), compile-time efficiency, and comparison with OpenSees and Kratos Multiphysics.

Each chapter includes the relevant mathematical theory with bibliographic references, the concrete C++ implementation in `fall_n`, and a discussion of the design rationale.

Contents

1 Simplex Finite Elements **6**
1.1 Reference Simplex . . . 6
1.2 Lagrange Shape Functions . . . 6
1.2.1 Linear Elements (Order 1) . . . 6
1.2.2 Quadratic Elements (Order 2) . . . 7
1.3 Shape Function Derivatives . . . 7
1.4 Isoparametric Mapping . . . 7
1.5 Subentity Topology: Faces of the Simplex . . . 8
1.5.1 Face Local Node Ordering (TET10) . . . 8
1.6 Gmsh Node Ordering for TET10 . . . 8
1.7 Implementation in `fall_n` . . . 9

2 Numerical Quadrature on Simplices **10**
2.1 Problem Statement . . . 10
2.2 Classical Symmetric Rules . . . 10
2.2.1 Centroid Rule (Degree 1) . . . 10
2.2.2 Hammer–Marlowe–Stroud Rules (HMS) . . . 10
2.2.3 Keast Rules . . . 11
2.2.4 Grundmann–Möller Rules . . . 11
2.3 Stroud Conical Product Rule . . . 11
2.3.1 Duffy Transform . . . 12
2.3.2 Product Quadrature with Gauss–Jacobi Nodes . . . 12
2.3.3 Cost Comparison . . . 13
2.4 Gauss–Jacobi Quadrature via the Golub–Welsch Algorithm . . . 13
2.4.1 Jacobi Polynomials . . . 13
2.4.2 The Golub–Welsch Algorithm . . . 13
2.4.3 Mapping to $[0, 1]$ . . . 14
2.5 Implementation: `StroudConicalProduct.hh` . . . 14
2.6 Default Quadrature Choices in `fall_n` . . . 15

3 Negative Weights and the Checkerboard Artefact **16**
3.1 Symptom: Element-Level Checkerboard Pattern . . . 16
3.2 Root Cause Analysis . . . 16
3.2.1 The Keast 5-Point Rule Weights . . . 17
3.2.2 TET10 Vertex Shape Functions at Quadrature Points . . . 17
3.2.3 The Denominator Catastrophe . . . 17
3.2.4 Consequence: Sign Flip or Division by Zero . . . 18
3.3 Why HEX27 Is Unaffected . . . 18
3.4 Resolution: Two Complementary Fixes . . . 18
3.4.1 Fix 1: Switch to an All-Positive-Weight Quadrature Rule . . . 18
3.4.2 Fix 2: Guard Against Negative Denominators . . . 18

3.4.3 Fix 3 (Recommended): SPR Volume-Weighted Averaging . . . 19
3.5 Summary . . . 19

4 Gauss-Point to Nodal Projection **20**
4.1 Lumped L^2 Projection . . . 20
4.1.1 Consistent L^2 Projection . . . 20
4.1.2 Row-Sum Lumping . . . 20
4.2 Volume-Weighted Averaging (SPR-Type) . . . 21
4.2.1 Motivation . . . 21
4.2.2 Algorithm . . . 21
4.2.3 Robustness Guarantee . . . 21
4.2.4 Comparison with Lumped L^2 . . . 22
4.3 Implementation: `spr_average_to_nodes()` . . . 22
4.4 Verification . . . 22

5 PETSc Patch: `section.c` Indexing Bug **23**
5.1 Context: DMPlex and PetscSection . . . 23
5.2 The Bug . . . 23
5.3 Manifestation in `fall_n` . . . 24
5.4 The Fix . . . 24
5.5 Verification . . . 24
5.6 Upstream Status . . . 24
5.7 Workaround from Application Code . . . 25
5.7.1 Workaround 1: Extend the Section Chart . . . 25
5.7.2 Workaround 2: Allow Dynamic Matrix Allocation . . . 25
5.7.3 Verification . . . 26

6 Implementation Summary and Design Decisions **27**
6.1 Architecture Overview . . . 27
6.2 Design Decisions . . . 27
6.2.1 Decision 1: HMS 4-Point as Default TET10 Quadrature . . . 27
6.2.2 Decision 2: Golub–Welsch for Gauss–Jacobi Quadrature . . . 28
6.2.3 Decision 3: SPR Volume-Weighted Averaging as Default Projection . . . 28
6.2.4 Decision 4: Gmsh TET10 Node Reordering . . . 29
6.2.5 Decision 5: PETSc Section Patch . . . 29
6.3 Nonlinear Showcase (`main.cpp`) . . . 29
6.4 Test Suite . . . 30
6.5 File Inventory . . . 31

7 Element Architecture: Design Patterns and Extensibility **32**
7.1 Guiding Principles . . . 32
7.2 Layer Decomposition . . . 32
7.2.1 Layer 1: Concrete Geometry Types . . . 33
7.2.2 Layer 2: ElementGeometry (Type-Erased Handle) . . . 33
7.2.3 Layer 3: Physical Elements . . . 35
7.2.4 Layer 4: Analysis Level . . . 35
7.3 FEM_Element: Top-Level Type Erasure . . . 35
7.4 Policy-Based Design: Customisation Points . . . 36
7.4.1 MaterialPolicy . . . 36
7.4.2 KinematicPolicy . . . 36
7.4.3 ElementPolicy (Storage Strategy) . . . 37
7.4.4 AssemblyPolicy (Beams) . . . 37

7.4.5 ConstitutiveStrategy (Material Update) . . . 38
7.5 Compile-Time Cost Analysis . . . 38
7.6 Comparison with OpenSees and Kratos . . . 39
7.6.1 Overview . . . 39
7.6.2 OpenSees . . . 39
7.6.3 Kratos Multiphysics . . . 40
7.6.4 Summary: Architectural Trade-offs . . . 40
7.7 Extensibility Guide: Adding a New Element Family . . . 41
7.8 Runtime Efficiency Considerations . . . 42
7.8.1 Virtual Dispatch Budget . . . 42
7.8.2 Cache-Friendly Layout . . . 42
7.8.3 DOF Index Caching . . . 42
7.8.4 Stack-Allocated Jacobians . . . 42

8 Generic Boundary and Initial Condition Infrastructure 43
8.1 Gap Analysis: the Legacy API . . . 43
8.2 Design Decisions and Trade-offs . . . 43
8.2.1 Compile-Time Polymorphism via Concepts . . . 43
8.2.2 Constraint Storage: Extending the Existing Map . . . 44
8.2.3 Non-Zero Dirichlet: Where Values Flow . . . 45
8.2.4 ModelState: Platform-Independent Snapshot . . . 45
8.3 Implementation . . . 45
8.3.1 NodeSelector.hh — Built-in Selectors . . . 45
8.3.2 Model.hh — New Constraint Methods . . . 46
8.3.3 Backward Compatibility . . . 46
8.4 Use-Case Walkthrough: Roller vs. Clamped Cantilever . . . 46
8.5 State Capture and Continuation Analysis . . . 47
8.5.1 ModelState Structure . . . 47
8.5.2 Capture Workflow . . . 48
8.5.3 Future: Initial Stress Integration . . . 48
8.6 Comparison with OpenSees and Kratos . . . 48
8.7 Extensibility Roadmap . . . 49
8.8 Test Suite . . . 49
8.9 Summary . . . 50

9 ElementGeometry Refactor 51
9.1 Problem Statement . . . 51
9.1.1 Broken value semantics . . . 51
9.1.2 Integration strategy mixed into concrete geometry headers . . . 51
9.1.3 Weak compile-time invariants . . . 51
9.2 Design Goals . . . 52
9.3 Refactor Overview . . . 52
9.3.1 Metadata moved under wrapper ownership . . . 52
9.3.2 Quadrature strategies extracted to dedicated headers . . . 52
9.3.3 A small quadrature concept . . . 53
9.4 Geometry and Measure . . . 53
9.5 API Hardening . . . 53
9.5.1 Restricting `detJ()` . . . 53
9.5.2 Fail-fast Gauss–Legendre tables . . . 53
9.5.3 Safer tensor-product constraints . . . 53
9.6 Materialised Integration Points . . . 54
9.7 Performance Discussion . . . 54

9.7.1 What did not change . . . 54
9.7.2 What did change . . . 54
9.8 What Was Deliberately Deferred . . . 54
9.9 Result . . . 54

10 Serendipity Element Families **56**
10.1 Why Serendipity? . . . 56
10.2 Mathematical Structure . . . 56
10.2.1 Tensor-product serendipity on the cube . . . 56
10.2.2 Why the tetrahedral "serendipity" family is different . . . 57
10.3 Implementation Structure . . . 57
10.3.1 Reference cells . . . 57
10.3.2 Concrete geometry . . . 58
10.3.3 ElementGeometry integration . . . 58
10.3.4 Quadrature choices . . . 58
10.4 VTK and Gmsh Support . . . 58
10.4.1 VTK . . . 58
10.4.2 Gmsh . . . 58
10.5 Validation Tests . . . 59
10.6 Benchmark Results . . . 59
10.7 Architectural Takeaways . . . 59
10.8 Future Work . . . 60

11 Domain Refactor Ontology **61**
11.1 Architectural Diagnosis . . . 61
11.2 Entity-by-Entity Semantics . . . 62
11.2.1 `Point<Dim>` . . . 62
11.2.2 `Vertex<Dim>` (proposed and introduced in this step) . . . 62
11.2.3 `Node<Dim, Storage>` . . . 62
11.2.4 `IntegrationPoint<dim>` . . . 63
11.2.5 `MaterialPoint<MaterialPolicy>` . . . 63
11.2.6 `SectionGeometry` . . . 63
11.2.7 `NodeSection<Dim, Geom>` . . . 64
11.2.8 `MaterialSection<MaterialPolicy>` . . . 64
11.2.9 `PetscSection` . . . 64
11.3 Coincidence Versus Identity . . . 64
11.3.1 Case 1: Gauss–Lobatto beam quadrature . . . 65
11.3.2 Case 2: mixed or assumed-strain formulations . . . 65
11.4 Proposed Ontology for the Refactor . . . 65
11.4.1 Layer 0: pure geometry/topology . . . 65
11.4.2 Layer 1: discrete field attachment . . . 66
11.4.3 Layer 2: sampling and constitutive sites . . . 66
11.4.4 Layer 3: structural section semantics . . . 66
11.5 Implications for `Domain` . . . 66
11.6 HPC and Compile-Time Guidance . . . 67
11.6.1 Prefer static structure in hot paths . . . 67
11.6.2 Keep `Domain` simple, not hyper-generic . . . 67
11.6.3 Avoid type collapse when roles overlap . . . 67
11.6.4 Recommended storage philosophy . . . 67
11.7 Migration Plan . . . 67
11.8 Conclusion . . . 68

12 Domain Vertex Transition **69**
12.1 Motivation . . . 69
12.2 Implemented Transition Design . . . 69
12.2.1 Canonical storage . . . 69
12.2.2 Node cache . . . 70
12.2.3 Cache invalidation rule . . . 70
12.3 Why This Preserves Performance . . . 70
12.3.1 No change to element hot loops . . . 70
12.3.2 Geometry-only consumers stop depending on nodes . . . 70
12.3.3 PETSc creation is now lazy . . . 71
12.4 Code-Level Changes . . . 71
12.5 What Remains Transitional . . . 71
12.6 Validation . . . 71
12.6.1 Domain/Vertex refactor test . . . 71
12.6.2 Regression coverage . . . 72
12.7 Architectural Consequences . . . 72
12.8 Next Step . . . 72
12.9 Conclusion . . . 72

13 Geometry-Only Element Binding **73**
13.1 Context . . . 73
13.2 Problem Statement . . . 73
13.3 Design Objective . . . 73
13.4 Implemented Refactor . . . 74
13.4.1 New dual binding inside concrete elements . . . 74
13.4.2 Wrapper-level support in `ElementGeometry` . . . 74
13.4.3 Key compatibility rule . . . 74
13.5 Domain-Level Consequences . . . 74
13.5.1 Geometry-first linking . . . 74
13.5.2 Boundary extraction no longer needs `node_p().id()` . . . 75
13.5.3 Sieve assembly remains analysis-oriented . . . 75
13.6 Why this is efficient . . . 75
13.6.1 No new runtime polymorphism in the hot path . . . 75
13.6.2 Compile-time impact . . . 75
13.7 Validation . . . 76
13.8 Architectural Interpretation . . . 76
13.9 Next Step . . . 76

Chapter 1

Simplex Finite Elements

This chapter describes the construction of Lagrangian finite elements on simplicial cells (triangles in 2D, tetrahedra in 3D) as implemented in `fall_n`. The development follows the classical Ciarlet definition [2] specialised to Lagrange interpolation on the reference simplex.

1.1 Reference Simplex

Definition 1.1 (Reference simplex)**.** The *reference simplex* in $\mathbb{R}^D$ is the convex hull

$$\mathcal{S}_D \;=\; \Big\{\boldsymbol{\xi} = (\xi_1, \ldots, \xi_D) \in \mathbb{R}^D \;\Big|\; \xi_i \geq 0,\; \sum_{i=1}^{D} \xi_i \leq 1\Big\}, \tag{1.1}$$

with *vertices*

$$\mathbf{v}_0 = \mathbf{0}, \qquad \mathbf{v}_i = \mathbf{e}_i \quad (i = 1, \ldots, D),$$

where $\mathbf{e}_i$ is the i-th standard basis vector.

The *barycentric coordinates* $(\lambda_0, \lambda_1, \ldots, \lambda_D)$ are defined as

$$\lambda_0 = 1 - \sum_{i=1}^{D} \xi_i, \qquad \lambda_i = \xi_i \quad (i = 1, \ldots, D). \tag{1.2}$$

They form a partition of unity, $\sum_{i=0}^{D} \lambda_i = 1$, and satisfy $\lambda_i \geq 0$ on $\mathcal{S}_D$.

The volume of the reference simplex is

$$|\mathcal{S}_D| = \frac{1}{D!}\,. \tag{1.3}$$

1.2 Lagrange Shape Functions

1.2.1 Linear Elements (Order 1)

The linear simplex has $D+1$ nodes (the vertices). The shape functions are simply the barycentric coordinates:

$$N_i(\boldsymbol{\xi}) = \lambda_i(\boldsymbol{\xi}), \qquad i = 0, 1, \ldots, D. \tag{1.4}$$

Since $\lambda_i \geq 0$ on $\mathcal{S}_D$ and $\lambda_i(\mathbf{v}_j) = \delta_{ij}$, these are the unique linear polynomials that interpolate the nodal values.

Concrete instances in `fall_n`:

- TRI3 ($D = 2$, 3 nodes): `SimplexElement<3,2,1>` (triangle in 3D ambient space, used for boundary faces)
- TET4 ($D = 3$, 4 nodes): `SimplexElement<3,3,1>`

1.2.2 Quadratic Elements (Order 2)

The quadratic simplex has

$$n_{\text{nodes}} = \binom{D+2}{2} = \frac{(D+1)(D+2)}{2} \tag{1.5}$$

nodes: the $D+1$ vertices plus $\binom{D+1}{2}$ edge midpoints.

Vertex shape functions. For vertex i ($i = 0, \ldots, D$):

$$\boxed{N_i(\boldsymbol{\xi}) = \lambda_i\,(2\lambda_i - 1)} \tag{1.6}$$

Remark 1.1 (Sign of vertex shape functions)**.** The vertex shape function $N_i = \lambda_i(2\lambda_i - 1)$ is *negative* whenever $0 < \lambda_i < \frac{1}{2}$. This region is non-empty in the interior of $\mathcal{S}_D$. In particular, interior quadrature points that are "far" from vertex i will have $\lambda_i \ll \frac{1}{2}$, making $N_i < 0$ at those points.

This fact is benign for stiffness matrix assembly (the bilinear form involves gradients, not function values). However, it becomes critical for lumped L^2 projection, as discussed in chapter 3.

Edge midpoint shape functions. For the midpoint of edge (i, j) with $0 \le i < j \le D$:

$$\boxed{N_{ij}(\boldsymbol{\xi}) = 4\,\lambda_i\,\lambda_j} \tag{1.7}$$

These are always non-negative on $\mathcal{S}_D$, since both $\lambda_i, \lambda_j \ge 0$.

Concrete instances:

- TRI6 ($D = 2$, 6 nodes): `SimplexElement<3,2,2>`
- TET10 ($D = 3$, 10 nodes): `SimplexElement<3,3,2>`

1.3 Shape Function Derivatives

Since $\lambda_0 = 1 - \xi_1 - \cdots - \xi_D$ and $\lambda_i = \xi_i$ for $i \ge 1$, the barycentric coordinate gradients (with respect to $\boldsymbol{\xi}$) are:

$$\frac{\partial \lambda_0}{\partial \xi_k} = -1, \qquad \frac{\partial \lambda_i}{\partial \xi_k} = \delta_{ik} \quad (i = 1, \ldots, D). \tag{1.8}$$

For the quadratic vertex shape function $N_i = \lambda_i(2\lambda_i - 1)$:

$$\frac{\partial N_i}{\partial \xi_k} = (4\lambda_i - 1)\,\frac{\partial \lambda_i}{\partial \xi_k}\,. \tag{1.9}$$

For the edge midpoint $N_{ij} = 4\lambda_i\lambda_j$:

$$\frac{\partial N_{ij}}{\partial \xi_k} = 4\Big(\lambda_j\,\frac{\partial \lambda_i}{\partial \xi_k} + \lambda_i\,\frac{\partial \lambda_j}{\partial \xi_k}\Big). \tag{1.10}$$

1.4 Isoparametric Mapping

The isoparametric map from the reference simplex $\mathcal{S}_D$ to a physical element $\Omega^e \subset \mathbb{R}^d$ (with $d \ge D$) is

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{I=1}^{n_{\text{nodes}}} N_I(\boldsymbol{\xi})\,\mathbf{x}_I, \tag{1.11}$$

where $\mathbf{x}_I \in \mathbb{R}^d$ are the physical nodal coordinates. The *Jacobian matrix* of the mapping is

$$J_{ij} = \frac{\partial x_i}{\partial \xi_j} = \sum_{I=1}^{n_{\text{nodes}}} \frac{\partial N_I}{\partial \xi_j} x_{I,i}\,, \qquad \mathbf{J} \in \mathbb{R}^{d\times D}. \tag{1.12}$$

When $d = D$ (volume elements), $\mathbf{J}$ is square and the differential volume element is $\mathrm{d}\Omega = |\det \mathbf{J}|\,\mathrm{d}\boldsymbol{\xi}$.

When $d > D$ (surface elements, e.g. TRI6 boundary faces in 3D), the surface measure is computed from the metric tensor:

$$\mathrm{d}S = \sqrt{\det(\mathbf{J}^T\mathbf{J})}\,\mathrm{d}\boldsymbol{\xi}. \tag{1.13}$$

Gradient transformation. The gradient of a scalar field in physical space is obtained from the reference-space gradient via

$$\frac{\partial N_I}{\partial x_j} = \sum_{k=1}^{D} \frac{\partial N_I}{\partial \xi_k} (J^{-1})_{kj}\,. \tag{1.14}$$

1.5 Subentity Topology: Faces of the Simplex

A D-simplex has $D+1$ faces (codimension-1 sub-simplices). The f-th face ($f = 0, \ldots, D$) is the sub-simplex *opposite* vertex f, spanned by all vertices *except* $\mathbf{v}_f$.

1.5.1 Face Local Node Ordering (TET10)

For a quadratic tetrahedron ($D = 3$, TET10, 10 nodes), each of the 4 triangular faces is a TRI6 sub-element. The face-to-global node mapping implemented in `SimplexCell<3,2>` is:

Face f	Opposite vertex	Global nodes
0	$\mathbf{v}_0$	(1, 2, 3, 5, 8, 7)
1	$\mathbf{v}_1$	(0, 3, 2, 6, 8, 5)
2	$\mathbf{v}_2$	(0, 1, 3, 4, 7, 6)
3	$\mathbf{v}_3$	(0, 2, 1, 5, 4, 6)

The outward normal direction follows from the right-hand rule applied to the first three (vertex) nodes of each face. This topology is critical for consistent surface traction application via Gauss quadrature on boundary faces.

1.6 Gmsh Node Ordering for TET10

Gmsh uses a different midpoint ordering than `fall_n` [6]. The Gmsh TET10 local ordering is:

Position	0	1	2	3	4	5	6	7	8	9
Gmsh	v_0	v_1	v_2	v_3	m_{01}	m_{12}	m_{02}	m_{03}	m_{13}	m_{23}
`fall_n`	v_0	v_1	v_2	v_3	m_{01}	m_{02}	m_{03}	m_{12}	m_{23}	m_{13}

The reordering applied in `GmshDomainBuilder.hh` maps Gmsh indices to `fall_n` indices via the permutation:

$$(0, 1, 2, 3, 4, 6, 7, 5, 9, 8). \tag{1.15}$$

This is implemented as:

```
index_ordering(node_tags, 0,1,2,3,4,6,7,5,9,8).data()
```

1.7 Implementation in `fall_n`

The simplex element hierarchy consists of three layers:

1. **`SimplexCell<D, Order>`** (`SimplexCell.hh`) — Purely geometric: reference coordinates, shape functions and their derivatives, face topology. All methods are `consteval` or `constexpr`, enabling compile-time evaluation.

2. **`SimplexElement<Dim, TopDim, Order>`** (`SimplexElement.hh`) — Combines a `SimplexCell` with an integration strategy. Template parameters:
   - `Dim`: ambient space dimension (e.g., 3 for 3D problems);
   - `TopDim`: topological dimension (D);
   - `Order`: Lagrange interpolation order.

3. **`ElementGeometry<ElemType, Integrator>`** (`ElementGeometry.hh`) — Manages the Jacobian, physical gradients, and integration; parameterised by element type and quadrature rule. The integrator is a duck-typed object exposing `num_integration_points`, `reference_integration_` and `weight(i)`.

This layered design allows the quadrature rule to be swapped independently of the element geometry, which was crucial for diagnosing and resolving the negative-weight problem (see chapters 2 and 3).

Chapter 2

Numerical Quadrature on Simplices

Numerical integration on simplicial domains is fundamentally different from integration on tensor-product cells (quadrilaterals, hexahedra). On tensor-product cells, one constructs n^D-point rules by taking the Cartesian product of n-point Gauss–Legendre rules in each direction. On simplices, no such natural product structure exists; special rules must be constructed [12, 10].

This chapter surveys the approaches available, documents the rules used in `fall_n`, and motivates the Stroud conical product as the preferred arbitrary-order method.

2.1 Problem Statement

We seek to approximate the integral

$$I[f] = \int_{\mathcal{S}_D} f(\boldsymbol{\xi})\,\mathrm{d}\boldsymbol{\xi} \approx \sum_{g=1}^{n_g} w_g\, f(\boldsymbol{\xi}_g), \tag{2.1}$$

where $\mathcal{S}_D$ is the reference D-simplex (1.1), with volume $|\mathcal{S}_D| = 1/D!$. A rule has *degree of exactness* p if $I[f]$ is computed exactly for all polynomials f of total degree $\leq p$.

Minimum requirements for finite elements. For a displacement-based finite element with polynomial order k and a constitutive law that is polynomial of degree m in the strains:

- Stiffness matrix: needs degree $\geq 2(k-1)m$ (often $2(k-1)$ for linear elasticity, where $m = 1$).
- Mass matrix (or L^2 projection): needs degree $\geq 2k$.
- For TET10 ($k = 2$, $D = 3$): stiffness needs degree ≥ 2; mass/projection needs degree ≥ 4.

2.2 Classical Symmetric Rules

2.2.1 Centroid Rule (Degree 1)

The single-point centroid rule:

$$\boldsymbol{\xi}_1 = \Big(\frac{1}{D+1}, \ldots, \frac{1}{D+1}\Big), \qquad w_1 = \frac{1}{D!}\,. \tag{2.2}$$

This integrates all linear polynomials exactly. It is adequate for constant- strain elements (TET4) when only the stiffness matrix is needed, but insufficient for quadratic elements.

2.2.2 Hammer–Marlowe–Stroud Rules (HMS)

The Hammer–Marlowe–Stroud [9] family provides low- order symmetric rules. The 4-point rule for the tetrahedron, used as the default for TET10 in `fall_n`, has:

n_g	Degree	Description
1	1	Centroid
4	2	Vertices of a smaller inscribed tetrahedron

The 4-point degree-2 rule uses barycentric coordinates:

$$\lambda = (\alpha, \beta, \beta, \beta) \quad \text{and permutations thereof,} \qquad \alpha = \frac{5-\sqrt{5}}{20},\ \beta = \frac{5+3\sqrt{5}}{20}. \tag{2.3}$$

All four weights are equal, $w_g = \frac{1}{4!} = \frac{1}{24}$, and are **strictly positive**.

Remark 2.1. In `fall_n`, this 4-point HMS rule is the *default* quadrature for TET10 elements (`default_simplex_rule<3,2>()` in `SimplexQuadrature.hh`). It was chosen over the 5-point Keast degree-3 rule precisely because all weights are positive — see chapter 3 for the detailed analysis.

2.2.3 Keast Rules

Keast [10] tabulated a series of tetrahedral rules up to degree 8. The most commonly cited are:

n_g	Degree	Min weight	Comment
1	1	$+$	Centroid
4	2	$+$	Same as HMS
5	3	$-$	$w_1 = -\frac{2}{15}$ at centroid
11	4	$+$	All positive
14	5	$+$	All positive
24	6	$-$	Contains negative weights

The 5-point degree-3 rule is particularly notable:

$$\text{Keast-5:} \quad w_1 = -\frac{2}{15}\,|\mathcal{S}_3| = -\frac{2}{15}\cdot\frac{1}{6} = -\frac{1}{45}, \qquad w_{2,\ldots,5} = \frac{3}{40}\,|\mathcal{S}_3| = \frac{1}{80}. \tag{2.4}$$

The centroid weight is *negative*. While this does not affect the accuracy of the quadrature rule (it still integrates degree-3 polynomials exactly), it causes severe problems in lumped L^2 projection on quadratic elements, as analysed in chapter 3.

2.2.4 Grundmann–Möller Rules

The Grundmann–Möller family [8] generates rules of arbitrary odd degree for any simplex dimension, using a combinatorial formula. The rule of index s has degree $2s+1$ with

$$n_g = \binom{D+s+1}{D+1}$$

points. Unfortunately, for $s \geq 1$ (degree ≥ 3), some weights are *guaranteed to be negative*. This makes Grundmann–Möller unsuitable as a general-purpose rule when positive weights are required.

2.3 Stroud Conical Product Rule

The Stroud conical product [12, 4] transforms the simplex integral into a product of 1D integrals via a *collapsed-coordinate* (Duffy) transformation, enabling the use of Gauss–Jacobi quadrature in each direction.

2.3.1 Duffy Transform

The transformation from *collapsed coordinates* $(t_1, \ldots, t_D) \in [0,1]^D$ to simplex coordinates $(\xi_1, \ldots, \xi_D) \in \mathcal{S}_D$ is defined recursively:

$$\boxed{\begin{aligned} &\xi_1 = t_1, \\ &\xi_k = t_k \prod_{j=1}^{k-1} (1 - t_j), \qquad k = 2, \ldots, D. \end{aligned}} \tag{2.5}$$

The Jacobian of this transformation is

$$\left| \frac{\partial \boldsymbol{\xi}}{\partial \mathbf{t}} \right| = \prod_{k=1}^{D-1} (1 - t_k)^{D-k}. \tag{2.6}$$

Therefore, the integral over the simplex becomes

$$\int_{\mathcal{S}_D} f(\boldsymbol{\xi}) \, \mathrm{d}\boldsymbol{\xi} = \int_0^1 \cdots \int_0^1 f(\boldsymbol{\xi}(\mathbf{t})) \prod_{k=1}^{D-1} (1 - t_k)^{D-k} \, \mathrm{d}t_1 \cdots \mathrm{d}t_D. \tag{2.7}$$

2.3.2 Product Quadrature with Gauss–Jacobi Nodes

The key insight is that each factor $(1 - t_k)^{D-k}$ depends only on t_k, so it can be *absorbed into* the 1D quadrature rule for direction k. Specifically, for direction k (0-indexed), we need a 1D quadrature rule on $[0,1]$ with weight function $(1-t)^{\beta_k}$, where

$$\beta_k = D - 1 - k, \qquad k = 0, 1, \ldots, D - 1. \tag{2.8}$$

- $k = 0$: $\beta = D - 1$ (maximal weight for the "lowest" direction).
- $k = D - 2$: $\beta = 1$ (linear weight).
- $k = D - 1$: $\beta = 0$ (plain Gauss–Legendre on $[0,1]$).

The integral (2.7) becomes

$$\int_{\mathcal{S}_D} f(\boldsymbol{\xi}) \, \mathrm{d}\boldsymbol{\xi} = \prod_{k=0}^{D-1} \left(\int_0^1 g_k(t_k) \, (1 - t_k)^{\beta_k} \, \mathrm{d}t_k \right), \tag{2.9}$$

where g_k encodes the dependence of f on t_k after integrating all other variables. Applying n-point Gauss–Jacobi quadrature on $[0,1]$ with weight $(1-t)^{\beta_k}$ to each direction gives a product rule with:

$$\boxed{n_g = n^D \text{ points}, \qquad \text{degree of exactness} = 2n - 1.} \tag{2.10}$$

Proposition 2.1 (All weights positive)**.** Since each 1D Gauss–Jacobi rule has strictly positive weights (the weight function $(1-t)^\beta$ is positive on $(0,1)$ for $\beta > -1$, and all Gauss–Jacobi weights are positive for any $\alpha, \beta > -1$), the product rule inherits positivity:

$$w_g = \prod_{k=0}^{D-1} w_{k,i_k} > 0 \qquad \forall \, g. \tag{2.11}$$

This is the fundamental advantage of the conical product over symmetric simplex rules.

2.3.3 Cost Comparison

Rule	Points	Degree	Weights
HMS 4-pt (TET)	4	2	All +
Keast 5-pt	5	3	**1 negative**
Keast 11-pt	11	4	All +
Conical $n=2$, $D=3$	8	3	All +
Conical $n=3$, $D=3$	27	5	All +
Conical $n=4$, $D=3$	64	7	All +

The conical product uses more points than optimal symmetric rules for the same degree, but it guarantees positivity and extends to *any* polynomial degree. For the TET10 stiffness matrix (degree ≥ 2), the 4-point HMS rule is sufficient and more economical; the conical product is available as a drop-in replacement when higher accuracy is needed.

2.4 Gauss–Jacobi Quadrature via the Golub–Welsch Algorithm

The 1D Gauss–Jacobi rules needed for the conical product are computed at library initialisation time using the Golub–Welsch algorithm [7].

2.4.1 Jacobi Polynomials

The Jacobi polynomials $P_n^{(\alpha,\beta)}(x)$ are orthogonal on $[-1,1]$ with respect to the weight function

$$w(x) = (1-x)^\alpha (1+x)^\beta, \qquad \alpha, \beta > -1. \tag{2.12}$$

They satisfy the three-term recurrence [13, 5]:

$$P_{n+1}^{(\alpha,\beta)}(x) = (A_n\, x + B_n)\, P_n^{(\alpha,\beta)}(x) - C_n\, P_{n-1}^{(\alpha,\beta)}(x), \tag{2.13}$$

where the coefficients A_n, B_n, C_n are explicit functions of n, α, β.

The *monic* Jacobi polynomials $\hat{p}_n$ satisfy

$$\hat{p}_{n+1}(x) = (x - a_n)\, \hat{p}_n(x) - b_n\, \hat{p}_{n-1}(x),$$

with recurrence coefficients [5]:

$$a_n = \frac{\beta^2 - \alpha^2}{(2n+\alpha+\beta)(2n+\alpha+\beta+2)}, \tag{2.14}$$

$$b_n = \frac{4n(n+\alpha)(n+\beta)(n+\alpha+\beta)}{(2n+\alpha+\beta)^2(2n+\alpha+\beta+1)(2n+\alpha+\beta-1)}. \tag{2.15}$$

2.4.2 The Golub–Welsch Algorithm

The Golub–Welsch algorithm [7] computes the n-point Gauss quadrature rule from the eigendecomposition of the *symmetric tridiagonal Jacobi matrix* $\mathbf{T} \in \mathbb{R}^{n\times n}$:

$$\mathbf{T} = \begin{pmatrix} a_0 & \sqrt{b_1} & & \\ \sqrt{b_1} & a_1 & \sqrt{b_2} & \\ & \ddots & \ddots & \ddots \\ & & \sqrt{b_{n-1}} & a_{n-1} \end{pmatrix}. \tag{2.16}$$

- The **eigenvalues** $\lambda_1, \ldots, \lambda_n$ of $\mathbf{T}$ are the *quadrature nodes* (zeros of the degree-n monic Jacobi polynomial).

- The **weights** are $w_i = \mu_0 \, v_{1,i}^2$, where $v_{1,i}$ is the first component of the i-th normalised eigenvector and μ_0 is the zeroth moment of the weight function:

$$\mu_0 = \int_{-1}^{1} (1-x)^\alpha (1+x)^\beta \, \mathrm{d}x = \frac{2^{\alpha+\beta+1} \, \Gamma(\alpha+1) \, \Gamma(\beta+1)}{\Gamma(\alpha+\beta+2)}. \tag{2.17}$$

Remark 2.2 (Why Golub–Welsch instead of Newton iteration)**.** An earlier implementation attempted to find the Jacobi polynomial zeros via a Newton iteration with Tricomi-type initial guesses. This approach failed for the $\alpha = D-1$ cases needed by the conical product (e.g., $\alpha = 2$ for $D = 3$, direction $k = 0$): the initial guesses were too inaccurate, and the iteration converged to *duplicate roots* instead of distinct nodes.

The Golub–Welsch algorithm is unconditionally robust: it requires no initial guesses and produces all nodes and weights simultaneously from a single eigenvalue decomposition. It is implemented using Eigen's `SelfAdjointEigenSolver`, which is $O(n^2)$ for the tridiagonal case.

2.4.3 Mapping to $[0, 1]$

The Gauss–Jacobi rule on $[-1, 1]$ with weight $(1-x)^\alpha$ is mapped to $[0, 1]$ via $x = 2t - 1$:

$$\int_0^1 f(t) \, (1-t)^\beta \, \mathrm{d}t = \frac{1}{2^{\beta+1}} \int_{-1}^{1} f\left(\frac{1+x}{2}\right) (1-x)^\beta \, \mathrm{d}x. \tag{2.18}$$

The nodes and weights transform as

$$t_i = \frac{1+x_i}{2}, \qquad w_i^{[0,1]} = \frac{w_i^{[-1,1]}}{2^{\beta+1}}. \tag{2.19}$$

This is implemented in `gauss_jacobi_01()` in `StroudConicalProduct.hh`.

2.5 Implementation: `StroudConicalProduct.hh`

The implementation resides in `src/numerics/numerical_integration/StroudConicalProduct.hh` and consists of three layers:

1. **`gauss_jacobi(n, α, β)`** — Returns an n-point Gauss–Jacobi rule on $[-1, 1]$ via Golub–Welsch. Uses Eigen's `SelfAdjointEigenSolver` on the tridiagonal Jacobi matrix.

2. **`stroud_conical_product<D>(n_per_dir)`** — Builds D one-dimensional Gauss–Jacobi rules (with $\beta_k = D - 1 - k$), takes their tensor product in collapsed coordinates, and maps to simplex coordinates via the Duffy transform (2.5).

3. **`ConicalProductIntegrator<TopDim, NPerDir>`** — Compile-time-sized integrator class. The rule is computed once at program startup (static `inline const` initialiser) and stored in a fixed-size `std::array`. Exposes the same interface as all other integrators in `fall_n`:
   - `static constexpr num_integration_points = NPerDir`$^{\texttt{TopDim}}$
   - `reference_integration_point(i)` returns a `std::span<const double>`
   - `weight(i)` returns w_i
   - `operator()(f)` evaluates $\sum w_i f(\boldsymbol{\xi}_i)$

2.6 Default Quadrature Choices in `fall_n`

Element	Rule	Points	Degree	Source
TET4	HMS 1-pt	1	1	`default_simplex_rule<3,1>()`
TET10	HMS 4-pt	4	2	`default_simplex_rule<3,2>()`
TRI3	HMS 1-pt	1	1	`default_simplex_rule<2,1>()`
TRI6	HMS 3-pt	3	2	`default_simplex_rule<2,2>()`
HEX27	Gauss 3^3	27	5	Tensor-product Gauss–Legendre

The conical product integrators (`ConicalProductIntegrator<3,2>`, `ConicalProductIntegrator<3,3>`, etc.) are available as drop-in replacements when higher-degree or higher-accuracy integration is needed, with the guarantee that all weights remain positive.

Chapter 3

Negative Weights and the Checkerboard Artefact

This chapter documents the diagnosis and resolution of a persistent *checkerboard* (or "diamond") artefact observed in nodal stress fields projected from Gauss points onto quadratic tetrahedral elements (TET10). The analysis traces the root cause to the interaction between negative quadrature weights and shape function values, and prescribes two complementary fixes.

3.1 Symptom: Element-Level Checkerboard Pattern

When visualising projected stress or strain fields in ParaView on TET10 meshes, the following pathology was observed:

- At *vertex* nodes, the projected field values had the *opposite sign* of the expected physical values.
- At *edge midpoint* nodes, the values were correct.
- The pattern repeated *at every element*, forming a regular alternating (checkerboard) structure.
- On HEX27 meshes, the same analysis pipeline produced smooth, correct fields.

3.2 Root Cause Analysis

The projected nodal values were computed via *lumped L^2 projection*:

$$\tilde{\sigma}_I = \frac{\displaystyle\sum_{e \in \mathcal{E}_I} \sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\, w_g\, |J_g|\, \sigma_g}{\displaystyle\sum_{e \in \mathcal{E}_I} \sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\, w_g\, |J_g|}, \tag{3.1}$$

where $\mathcal{E}_I$ is the set of elements sharing node I, N_I is the global shape function, and σ_g is the Gauss-point value.

For the projection (3.1) to produce meaningful results, the denominator must be **strictly positive** for every node. We will show that this condition *fails* at vertex nodes of TET10 when the Keast 5-point rule is used.

3.2.1 The Keast 5-Point Rule Weights

The Keast degree-3 rule for the tetrahedron has 5 points (section 2.2.3):

$$\begin{aligned} &\text{Point 1 (centroid):} && w_1 = -\frac{2}{15}\,|\mathcal{S}_3| = -\frac{1}{45} \approx -0.02222, \\ &\text{Points 2–5 (vertices):} && w_g = +\frac{3}{40}\,|\mathcal{S}_3| = +\frac{1}{80} = 0.01250. \end{aligned} \tag{3.2}$$

The centroid weight w_1 is **negative**.

3.2.2 TET10 Vertex Shape Functions at Quadrature Points

The vertex shape function for node i is (cf. eq. (1.6)):

$$N_i(\boldsymbol{\xi}) = \lambda_i\,(2\lambda_i - 1).$$

At the 5 Keast quadrature points:

At the centroid ($\lambda_i = 1/4$ for all i):

$$N_i = \tfrac{1}{4}(2 \cdot \tfrac{1}{4} - 1) = \tfrac{1}{4}(-\tfrac{1}{2}) = -\tfrac{1}{8}.$$

At the vertex-orbit points: Each vertex-orbit point has one "dominant" barycentric coordinate near 0.5854 and three "small" ones near 0.1382.

For the vertex i that is *dominant* at point g:

$$N_i \approx 0.5854 \times (2 \times 0.5854 - 1) = 0.5854 \times 0.1708 \approx 0.1000.$$

For a vertex i that is *not dominant*:

$$N_i \approx 0.1382 \times (2 \times 0.1382 - 1) = 0.1382 \times (-0.7236) \approx -0.1000.$$

3.2.3 The Denominator Catastrophe

Consider the denominator of (3.1) for a *vertex* node I, restricted to a single element:

$$D_I = \sum_{g=1}^{5} N_I(\boldsymbol{\xi}_g)\,w_g\,|J_g|. \tag{3.3}$$

Assuming approximately constant $|J_g| \approx |J|$ within the element:

$$D_I \approx |J|\big[\underbrace{N_I(\boldsymbol{\xi}_1)\,w_1}_{\text{centroid: } (-1/8)(-1/45)>0} + N_I(\boldsymbol{\xi}_2)\,w_2 + \cdots + N_I(\boldsymbol{\xi}_5)\,w_5\big]. \tag{3.4}$$

The centroid contribution is *positive* (negative times negative), but the vertex-orbit contributions produce a mix of positive and negative terms. The crucial issue is that at the three vertex-orbit points where $N_I < 0$ (because $\lambda_I < 1/2$), the *positive* weight $w_g > 0$ creates a negative contribution, while at the one vertex-orbit point where $N_I > 0$ (the "own" vertex direction), the positive weight creates a positive contribution.

When these are summed, the negative contributions can *overwhelm* the positive ones, making $D_I < 0$ or $D_I \approx 0$.

3.2.4 Consequence: Sign Flip or Division by Zero

$$D_I \le 0 \implies \tilde{\sigma}_I = \frac{(\text{numerator})}{D_I} \begin{cases} \text{has wrong sign} & \text{if } D_I < 0, \\ \text{diverges} & \text{if } D_I = 0. \end{cases} \tag{3.5}$$

In practice, the code included a guard $|D_I| < \epsilon \implies \tilde{\sigma}_I = 0$. This zeroed out vertex values while midpoint values (which have $N_{ij} = 4\lambda_i\lambda_j \ge 0$ everywhere) remained correct. The result was the characteristic checkerboard pattern:

Node type	Denominator D_I	Projected value
Vertex	≤ 0 or ≈ 0	0 or wrong sign
Midpoint	> 0	Correct

3.3 Why HEX27 Is Unaffected

HEX27 uses the tensor-product $3 \times 3 \times 3$ Gauss–Legendre rule (27 points), which has **all positive weights**. Furthermore, the Lagrangian shape functions on the reference cube $[-1, 1]^3$ are products of 1D Lagrange polynomials, and while they can go negative, they do so in a balanced way that keeps the lumped denominator positive. The combination of all-positive weights and the tensor-product structure prevents the denominator catastrophe.

3.4 Resolution: Two Complementary Fixes

Two independent corrections were applied:

3.4.1 Fix 1: Switch to an All-Positive-Weight Quadrature Rule

The root cause is the negative weight $w_1 = -2/15\,|\mathcal{S}_3|$ in the Keast 5-point rule. Replacing it with the **HMS 4-point degree-2 rule**, which has all positive weights (section 2.2.2), eliminates the negative-weight pathway entirely.

In `SimplexQuadrature.hh`, the default for TET10 was changed:

```
template<>
inline constexpr auto default_simplex_rule<3, 2>() {
    // Was: make_simplex_rule_3d_5()  [Keast 5-pt, w1 = -2/15]
    // Now: make_simplex_rule_3d_4()  [HMS 4-pt, all w > 0]
    return make_simplex_rule_3d_4();
}
```

Remark 3.1. The HMS 4-point rule has degree 2, one degree lower than the Keast 5-point rule (degree 3). For the TET10 stiffness matrix, degree 2 is the minimum requirement ($2(k-1) = 2$ for $k = 2$), so accuracy is maintained for linear elasticity. For the mass matrix or high-order nonlinear constitutive models, the conical product (`ConicalProductIntegrator<3,2>`, 8 points, degree 3) is available as a higher-degree drop-in replacement with all positive weights.

3.4.2 Fix 2: Guard Against Negative Denominators

As a defensive measure, the lumped L^2 projection code was modified to use the absolute value of the denominator:

```
denominator = std::abs(raw_denominator);
if (denominator < epsilon) denominator = epsilon;
```

This ensures that even if a future quadrature rule with negative weights is inadvertently used, the projection produces a *smoothed* (not sign-flipped) field. The `std::abs` guard does not affect the result when all weights are positive, since the denominator is then guaranteed positive.

3.4.3 Fix 3 (Recommended): SPR Volume-Weighted Averaging

The most robust solution is to bypass lumped L^2 projection entirely and use **volume-weighted nodal averaging** — a simplified form of the Superconvergent Patch Recovery (SPR) technique [14]. This is discussed in detail in chapter 4.

3.5 Summary

The checkerboard artefact results from the interaction of three factors:

1. **Negative quadrature weight** in the Keast 5-point rule ($w_1 = -2/15\,|\mathcal{S}_3|$).
2. **Negative vertex shape functions** of TET10 at interior quadrature points ($N_i = \lambda_i(2\lambda_i - 1) < 0$ when $\lambda_i < 1/2$).
3. **Lumped L^2 projection** with a denominator that sums signed products $N_I \cdot w_g$, allowing cancellation to zero or sign reversal at vertex nodes.

Any one of these factors being absent would prevent the artefact. The chosen resolution addresses all three:

- Factor 1: eliminated by switching to HMS 4-point (all $w_g > 0$);
- Factor 3: eliminated by switching to SPR averaging (no denominator subtraction possible).
- Factor 2: is intrinsic to quadratic elements and cannot be changed, but it is rendered harmless by the other two fixes.

Chapter 4

Gauss-Point to Nodal Projection

In displacement-based finite elements, the primary unknowns (displacements) are nodal. Derived quantities — strains, stresses, internal variables — are computed at *Gauss (integration) points* inside each element. For post-processing and visualisation, these quantities must be *projected* or *averaged* to the mesh nodes so that they can be rendered as continuous fields in VTK/ParaView.

This chapter compares the three projection methods implemented in `fall_n` and justifies the choice of volume-weighted averaging as the default for production use.

4.1 Lumped L^2 Projection

4.1.1 Consistent L^2 Projection

The mathematically rigorous approach minimises the L^2 norm of the error between the discrete Gauss-point field and a continuous nodal field. Let σ^h be the Gauss-point field and $\tilde{\sigma}^h = \sum_I \tilde{\sigma}_I N_I$ the nodal projection. The consistent L^2 projection solves:

$$\mathbf{M}\,\tilde{\boldsymbol{\sigma}} = \mathbf{f}, \tag{4.1}$$

where $\mathbf{M}$ is the global mass matrix ($M_{IJ} = \int_\Omega N_I N_J \,\mathrm{d}\Omega$) and $\mathbf{f}$ is the right-hand side ($f_I = \int_\Omega N_I \sigma^h \,\mathrm{d}\Omega$).

This requires solving a global linear system, which is expensive and may introduce oscillations at element boundaries (Gibbs-like effects) for discontinuous fields.

4.1.2 Row-Sum Lumping

The standard shortcut is to *lump* the mass matrix by replacing it with its row sums on the diagonal:

$$\tilde{M}_{II} = \sum_J M_{IJ} = \int_\Omega N_I \sum_J N_J \,\mathrm{d}\Omega = \int_\Omega N_I \,\mathrm{d}\Omega, \tag{4.2}$$

using the partition of unity $\sum_J N_J = 1$. The lumped projection then becomes a local (node-by-node) average:

$$\tilde{\sigma}_I = \frac{\displaystyle\int_\Omega N_I\,\sigma^h \,\mathrm{d}\Omega}{\displaystyle\int_\Omega N_I \,\mathrm{d}\Omega} \approx \frac{\displaystyle\sum_{e\in\mathcal{E}_I}\sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\,w_g\,|J_g|\,\sigma_g}{\displaystyle\sum_{e\in\mathcal{E}_I}\sum_{g=1}^{n_g} N_I(\boldsymbol{\xi}_g)\,w_g\,|J_g|}. \tag{4.3}$$

The denominator is the "nodal volume" contributed by the quadrature rule. For it to be meaningful, we need

$$D_I = \sum_{e\in\mathcal{E}_I}\sum_g N_I(\boldsymbol{\xi}_g)\,w_g\,|J_g| > 0 \qquad \forall\, I. \tag{4.4}$$

As demonstrated in chapter 3, this condition *fails* at vertex nodes of TET10 when the Keast 5-point rule is used, due to the interaction of negative weights with negative vertex shape function values.

Implementation. The lumped L^2 projection is implemented in `l2_project_to_nodes()` in `VTKModelExporter.hh`. After the checkerboard diagnosis, the code was patched with an `std::abs` guard on the denominator:

```
double denom = std::abs(node_lump[I]);
if (denom < 1e-14) denom = 1e-14;
node_val[I] /= denom;
```

4.2 Volume-Weighted Averaging (SPR-Type)

4.2.1 Motivation

The Superconvergent Patch Recovery (SPR) method of Zienkiewicz and Zhu [14, 15] fits a polynomial to the Gauss-point values in a *patch* of elements surrounding each node and evaluates the polynomial at the node. In its full form, SPR requires solving local least-squares problems.

For production post-processing where visual smoothness (rather than superconvergent error estimation) is the primary goal, a simpler and even more robust approach suffices: **volume-weighted element averaging**.

4.2.2 Algorithm

For each node I, compute the nodal field as a weighted average of the element centroid values:

$$\boxed{\tilde{\sigma}_I = \frac{\sum_{e \in \mathcal{E}_I} V_e \, \bar{\sigma}_e}{\sum_{e \in \mathcal{E}_I} V_e},} \tag{4.5}$$

where:

- $\mathcal{E}_I$ is the set of elements sharing node I;
- V_e is the volume of element e (computed via quadrature: $V_e = \sum_g w_g |J_g|$);
- $\bar{\sigma}_e$ is the element-average value, computed as the weighted mean of Gauss-point values:

$$\bar{\sigma}_e = \frac{\sum_g w_g \, |J_g| \, \sigma_g}{\sum_g w_g \, |J_g|}. \tag{4.6}$$

4.2.3 Robustness Guarantee

Proposition 4.1 (Unconditional positivity)**.** All weights in the averaging formula (4.5) are element volumes $V_e > 0$, which are strictly positive for any non-degenerate element regardless of the quadrature rule used for stiffness integration. Even if the quadrature rule contains negative weights, the element volume $V_e = \sum_g w_g |J_g|$ remains positive (it is the exact volume when the quadrature degree is sufficient to integrate polynomials of the Jacobian's degree).

Therefore, SPR volume-weighted averaging is **unconditionally robust**: the denominator is always positive, and no sign flip or division by zero can occur at any node, regardless of element type or quadrature rule.

4.2.4 Comparison with Lumped L^2

Property	Lumped L^2	Volume-weighted avg.
Denominator sign	Can be ≤ 0	Always > 0
Sensitivity to $w_g < 0$	High (catastrophic)	None
Accuracy order	Optimal ($O(h^{k+1})$)	$O(h^k)$
Superconvergent?	No	No
Computational cost	$O(n_e \cdot n_g)$	$O(n_e \cdot n_g)$
Requires global solve?	No	No

For visualisation purposes, the $O(h^k)$ vs. $O(h^{k+1})$ accuracy difference is negligible. The robustness advantage of volume-weighted averaging is decisive.

4.3 Implementation: `spr_average_to_nodes()`

The volume-weighted averaging is implemented in two methods in `VTKModelExporter.hh`:

`spr_average_to_nodes()` Computes a single scalar field σ:

1. For each element e, compute V_e and $\bar{\sigma}_e$ from Gauss-point data.
2. For each node I of element e, accumulate:

$$\text{numerator}[I] \mathrel{+}= V_e \cdot \bar{\sigma}_e, \tag{4.7}$$

$$\text{denominator}[I] \mathrel{+}= V_e. \tag{4.8}$$

3. After all elements: $\tilde{\sigma}_I = \text{numerator}[I] \,/\, \text{denominator}[I]$.

`compute_material_fields_spr()` Applies `spr_average_to_nodes()` to all material fields: strain tensor (6 components), stress tensor (6 components), plastic strain tensor (6 components), and equivalent plastic strain (1 scalar). This is the method called in the updated `main.cpp`.

4.4 Verification

The SPR volume-weighted averaging was verified on the cantilever benchmark ($10 \times 0.4 \times 0.8$ box, $E = 200$, $\nu = 0.3$):

- **HEX27 mesh** (20 elements, 315 nodes): Both lumped L^2 and SPR produce smooth, visually identical stress fields.
- **TET10 mesh** (~ 1329 elements, 2840 nodes):
  - Lumped L^2 with Keast-5: **checkerboard at vertices**.
  - Lumped L^2 with HMS-4 + `abs` guard: smooth, correct.
  - SPR volume-weighted: smooth, correct.
- The main showcase (`main.cpp`) now uses `compute_material_fields_spr()` by default for all 14 cases (7 formulations $\times$ 2 element types).

Chapter 5

PETSc Patch: `section.c` Indexing Bug

This chapter documents a bug in PETSc 3.24.5 [1] that affects higher-order finite elements whose DMPlex point numbering does not start at zero. The bug was discovered during the development of simplex element support in `fall_n` and confirmed to be an upstream issue.

5.1 Context: DMPlex and PetscSection

PETSc's `DMPlex` represents an unstructured mesh as a directed acyclic graph (DAG) of *points*: vertices, edges, faces, and cells, numbered from 0 to $N - 1$ (though the numbering of each *stratum* may start at a non-zero offset).

A `PetscSection` associates degrees of freedom (DOFs) with mesh points. It uses an *atlas* (`s->atlasDof[]`) indexed relative to the chart [`pStart`, `pEnd`), and a *constraint* mechanism to impose Dirichlet boundary conditions by specifying which DOFs at a point are constrained.

The function `PetscSectionSetConstraintIndices()` sets the constrained DOF indices at a given point. Its implementation reads:

```
PetscCall(PetscSectionCheckConstraints_Private(s));
/* Bug: should subtract pStart to get the atlas offset */
if (s->atlasDof[point]) {               // <-- HERE
    PetscInt cdof;
    PetscCall(PetscSectionGetConstraintDof(s, point, &cdof));
    // ...
}
```

5.2 The Bug

In `src/sys/utils/section.c`, at approximately line 2997, the code reads:

```
if (s->atlasDof[point]) {
```

but the atlas array is indexed relative to `pStart`, so the correct access is:

```
if (s->atlasDof[point - s->pStart]) {
```

When `pStart = 0` (as happens for vertex stratum when vertices are numbered from zero), the two expressions are identical and the bug is invisible. However, when a mesh's vertex stratum starts at a non-zero offset — which occurs in `fall_n` because Gmsh-imported meshes place cells before vertices in the DAG, giving the vertex stratum `pStart` = 8 (for 8 hex cells) or larger — the raw `point` index overflows the atlas array, reading uninitialised memory.

5.3 Manifestation in `fall_n`

The bug manifests during the application of Dirichlet boundary conditions via `PetscSectionSetConstraintInd`
Specifically:

- For HEX27 meshes with 20 cells, the vertex stratum starts at `pStart` ≥ 20.
- When setting constraints on vertex DOFs, the index `s->atlasDof[point]` reads from memory at offset 20+ instead of offset 0+.
- The read value is unpredictable: it may be zero (causing the constraints to be silently skipped) or non-zero (proceeding correctly by accident).
- The symptom was a segmentation fault or incorrect constraint application on some meshes but not others, depending on memory layout.

5.4 The Fix

The fix is a one-line change in `src/sys/utils/section.c`:

```
// Before (buggy):
if (s->atlasDof[point]) {

// After (fixed):
if (s->atlasDof[point - s->pStart]) {
```

This was applied to the local PETSc installation at:

```
/home/sechavarriam/MyLibs/petsc-3.24.5/src/sys/utils/section.c
```

After applying the patch, PETSc was rebuilt and all `fall_n` constraint tests passed on both HEX27 and TET10 meshes.

5.5 Verification

The fix was verified by:

1. Running the debug output in `fall_n`'s boundary condition application code, which prints:

   ```
   [SIEVE-DBG] section_ptr=0x... chart=[9152,23629)
   [SIEVE-DBG] All 14477 section DOFs OK
   [BC-DBG] Constraints count: 145
   [BC-DBG] plex_id=9152 section_dof=3 constraint_size=3
     ...
   ```

   confirming that all 145 Dirichlet-constrained vertices are correctly handled with `pStart` $= 9152$.
2. Comparing the Case 1 (SmallStrain + Linear Elastic) displacement field on HEX27 against the expected Timoshenko beam solution: the tip displacement matches the analytical result to within the expected discretisation error.
3. Running all 7 nonlinear cases on both HEX27 and TET10 meshes without segfaults or constraint errors.

5.6 Upstream Status

As of PETSc 3.24.5, this bug is present in the release source. It has been reported to the PETSc developers. Since the bug only affects codes that:

- use `PetscSectionSetConstraintIndices` directly (rather than through higher-level `DMPlexSetBoundary`
  and
- have a `PetscSection` chart with `pStart` > 0,

it may not have been triggered by the PETSc test suite, which may use meshes with vertex-first ordering (`pStart` = 0 for the vertex stratum).

5.7 Workaround from Application Code

In parallel with the upstream patch, we discovered two application-level workarounds that avoid modifying PETSc source entirely. Both were implemented in `fall_n` and verified on all mesh types.

5.7.1 Workaround 1: Extend the Section Chart

The root cause is that `PetscSectionSetConstraintIndices()` accesses `s->atlasDof[point]` without subtracting `s->pStart`, so it reads out of bounds whenever the chart does not start at zero.

Symptom. For the TET4 mesh (9152 elements, 2119 nodes), the `DMPlex` point numbering places cells at $[0, 9152)$ and vertices at $[9152, 11271)$. When the section chart is set to the vertex stratum $[\texttt{vStart}, \texttt{vEnd}) = [9152, 11271)$, the atlas has only $11271 - 9152 = 2119$ entries. The buggy code accesses index 9152 into a 2119-entry array — a severe out-of-bounds read.

The read value is unpredictable: it reports `dof = 123` (instead of 3) for constrained nodes, producing 604 PETSc error lines of the form:

```
[0]PETSC ERROR: Point 9152 dof 123,
    invalid constraint index[9]: 24587
```

More critically, the constraint indices are *not applied*, and the system produces a singular matrix (all Dirichlet DOFs are unconstrained), giving `|u| = 0.000000` for TET4.

Fix. In `Model::set_sieve_layout()`, set the section chart to cover the full DM chart $[\texttt{pStart}, \texttt{pEnd})$ instead of only the vertex stratum $[\texttt{vStart}, \texttt{vEnd})$:

```
// Before (triggers PETSc bug when vStart > 0):
PetscSectionSetChart(dof_section, vStart, vEnd);

// After (workaround: atlas covers all points):
PetscSectionSetChart(dof_section, pStart, pEnd);
```

With the full chart, `pStart = 0` always, so the buggy raw index `s->atlasDof[point]` coincides with the correct adjusted index `s->atlasDof[point - 0]`. Element points (cells) simply carry 0 DOFs.

The cost is a marginally larger atlas array (8 bytes per cell point), which is negligible relative to the mesh data.

Source. `src/model/Model.hh`, method `set_sieve_layout()`.

5.7.2 Workaround 2: Allow Dynamic Matrix Allocation

A secondary class of errors surfaces on large unstructured TET4 meshes: PETSc reports thousands of *"New nonzero at (i,j) caused a malloc"* messages during matrix assembly. These occur because `DMCreateMatrix()`'s internal adjacency walk (`DMPlexPreallocateOperator`) underestimates the sparsity pattern for cell-vertex sieves of unstructured simplex meshes.

Diagnosis. On the TET4 mesh (9152 elements, 2119 nodes), analysis confirmed that error DOF pairs correspond to nodes that *do not share any element* in the mesh. For example, node 172 (9 neighbours in the mesh graph) was reported as needing an entry with node 1358 — with zero shared elements. This is a PETSc preallocation-algorithm issue, not a `fall_n` sieve-setup error.

The results were still correct because PETSc dynamically allocates the missing entries (just slower, with error messages in debug builds).

Fix. After each `DMCreateMatrix()` call, tell PETSc to silently allocate missing entries instead of reporting errors:

```
DMCreateMatrix(dm, &K);
MatSetOption(K, MAT_NEW_NONZERO_ALLOCATION_ERR, PETSC_FALSE);
```

This was applied to all matrix creation sites:

File	Method	Matrix
`Model.hh`	`setup_matrix()`	`Kt_`
`LinearAnalysis.hh`	`setup_matrix_sizes()`	`K`
`NLAnalysis.hh`	SNES Jacobian callback	`J`
`DynamicAnalysis.hh`	`setup()`	`M_, C_, J_`
`DynamicAnalysis.hh`	`assemble_initial_stiffness`	`K0_`

5.7.3 Verification

After applying both workarounds, `fall_n` runs with **zero** PETSc error messages on all four mesh types. The results table:

Model	Nodes	Elems	$\lvert u\rvert_{\max}$	$\lvert u_y\rvert_{\max}$	Error (%)
Timoshenko $N = 3$ (ref.)	11	5	6.4518	6.2578	0.00
HEX8 (30×6×3)	868	540	4.8293	4.8293	25.15
HEX27 (30×6×3)	5551	540	6.2148	6.2148	3.67
TET4 (Fine, 4-pt)	2119	9152	3.5079	3.5079	45.63
TET10 (Fine, 27-pt)	4914	2475	6.2173	6.2173	3.64

The TET4 result (3.51, 45.6% error) now correctly reflects the well-known volumetric locking and stiffness overestimation of linear tetrahedra under bending. Previously, the section bug caused `|u| = 0.000` (100% error) because boundary conditions were silently dropped.

The TET10 (3.64% error) and HEX27 (3.67% error) results match the Timoshenko beam reference to within the expected discretisation error for second-order elements.

Chapter 6

Implementation Summary and Design Decisions

This chapter collects the key design decisions made during the implementation of simplex elements in `fall_n`, with forward references to the relevant source files and rationale grounded in the analysis of the preceding chapters.

6.1 Architecture Overview

`fall_n` is a C++23 nonlinear finite element library. Its element hierarchy is fully *compile-time polymorphic*: element types, quadrature rules, constitutive models, and kinematic formulations are all resolved through templates without virtual dispatch.

Layer	Header	Responsibility
Geometry	`SimplexCell.hh`	Reference coords, basis, topology
Element	`SimplexElement.hh`	Iso-parametric map + integrator
Integration	`SimplexQuadrature.hh`	Classical symmetric rules
	`StroudConicalProduct.hh`	Arbitrary-order conical product
Mesh I/O	`GmshDomainBuilder.hh`	Gmsh → `Domain`
Model	`Model.hh`	DOF management, BC application
Post-processing	`VTKModelExporter.hh`	Gauss → node projection, VTU

6.2 Design Decisions

6.2.1 Decision 1: HMS 4-Point as Default TET10 Quadrature

Context
TET10 elements require at least degree-2 quadrature for the stiffness matrix. The Keast 5-point rule (degree 3) was the initial default, following the common "one degree above minimum" heuristic.

Problem
The Keast 5-point rule has $w_1 = -2/15\,|\mathcal{S}_3|$ at the centroid, causing the lumped L^2 projection to produce checkerboard artefacts (chapter 3).

Decision
Switch to HMS 4-point (degree 2) as the default. Degree 2 is the *exact minimum* for the TET10 stiffness matrix; it provides no excess integration accuracy but avoids the negative-weight problem entirely.

Alternative considered
Conical product $n = 2$ (8 points, degree 3, all positive). This was rejected as the *default* because it uses twice as many points as HMS 4, but it is available as `ConicalProductIntegrator<3,2>` for use cases requiring degree 3.

Source
`SimplexQuadrature.hh`, function `default_simplex_rule<3,2>()`.

6.2.2 Decision 2: Golub–Welsch for Gauss–Jacobi Quadrature

Context
The Stroud conical product in 3D requires Gauss–Jacobi quadrature with parameters up to $\alpha = D - 1 = 2$, $\beta = 0$.

Problem
An initial implementation used Newton iteration on the Jacobi polynomial (with Tricomi / asymptotic initial guesses). For $\alpha = 2$ and $n \geq 3$, the iteration converged to duplicate roots because the initial guesses were insufficiently accurate for the distorted zero distribution of high-α Jacobi polynomials.

Decision
Replace Newton iteration entirely with the Golub–Welsch eigenvalue algorithm (section 2.4). This algorithm is unconditionally robust for all valid $\alpha, \beta > -1$ and any n, since it requires no initial guesses.

Implementation
Uses Eigen's `SelfAdjointEigenSolver` on the $n \times n$ symmetric tridiagonal Jacobi matrix. Cost is $O(n^2)$; since $n \leq 10$ in practice, this is negligible.

Source
`StroudConicalProduct.hh`, function `gauss_jacobi()`.

6.2.3 Decision 3: SPR Volume-Weighted Averaging as Default Projection

Context
Gauss-point fields must be projected to nodes for VTU export.

Problem
Lumped L^2 projection is sensitive to negative quadrature weights (chapter 3). Even with HMS 4-point (all positive weights), future element types or user-supplied quadrature rules could reintroduce negative weights.

Decision
Use volume-weighted averaging (`spr_average_to_nodes()`) as the default production projection method. This method is unconditionally robust because all weights are element volumes (chapter 4).

Trade-off
Volume-weighted averaging is $O(h^k)$ vs. $O(h^{k+1})$ for lumped L^2, but this one-order accuracy difference is irrelevant for visualisation at practical mesh sizes.

Source
`VTKModelExporter.hh`, methods `spr_average_to_nodes()` and `compute_material_fields_spr()`.

6.2.4 Decision 4: Gmsh TET10 Node Reordering

Context
Gmsh's local node ordering for 10-node tetrahedra differs from `fall_n`'s ordering at the edge midpoints.

Problem
Without reordering, the Jacobian mapping is incorrect, producing negative determinants and garbage results.

Decision
Apply the permutation $(0, 1, 2, 3, 4, 6, 7, 5, 9, 8)$ to map Gmsh indices to `fall_n` indices. This is done at mesh import time in `GmshDomainBuilder.hh`.

Verification
The simplex geometry tests (`test_element_geometry.cpp`) verify positive Jacobian determinants and correct isoparametric mapping for both TET4 and TET10 elements imported from Gmsh.

Source
`GmshDomainBuilder.hh`, case 11 (TET10).

6.2.5 Decision 5: PETSc Section Patch

Context
`fall_n` uses PETSc's DMPlex for distributed mesh management and `PetscSection` for DOF layout.

Problem
A bug in `PetscSectionSetConstraintIndices()` reads the atlas array with an incorrect index when `pStart` > 0 (chapter 5).

Decision
Apply a local one-line patch: `s->atlasDof[point]` $\rightarrow$ `s->atlasDof[point - s->pStart]`.

Impact
Without this patch, Dirichlet boundary conditions are silently incorrect on meshes where the vertex stratum does not start at point 0.

Source
PETSc 3.24.5, `src/sys/utils/section.c`, line $\sim$2997.

6.3 Nonlinear Showcase (`main.cpp`)

The main executable exercises 7 constitutive/kinematic formulations on both HEX27 and TET10 meshes (14 cases total) on a cantilever beam:

Case	Kinematics	Constitutive	Load	Steps
1	SmallStrain	Linear elastic	0.05	1 (linear)
2	TotalLagrangian	SVK	0.05	1
3	TotalLagrangian	SVK	0.50	5
4	TotalLagrangian	Neo-Hookean	0.50	5
5	UpdatedLagrangian	SVK	0.05	1
6	UpdatedLagrangian	Neo-Hookean	0.50	5
7	SmallStrain	J2 plasticity	1.00	20

Material parameters: $E = 200$, $\nu = 0.3$, $\sigma_y = 0.250$, $H = 10.0$.

Geometry: $[0, 10] \times [0, 0.4] \times [0, 0.8]$. Fixed face at $x = 0$, traction on face at $x = 10$.

Meshes:

- `data/input/box.msh`: HEX27 (20 elements, 315 nodes).
- `data/input/box_tet10.msh`: TET10 ($\sim$1329 elements, 2840 nodes).

Post-processing: All cases use `compute_material_fields_spr()` for Gauss-to-node projection, producing dual VTU outputs (mesh + Gauss cloud) in `data/output/`.

6.4 Test Suite

The simplex-related tests in `tests/test_simplex.cpp` include 49 test cases covering:

- Node count, reference coordinates, and shape function values for TET4 and TET10.
- Barycentric coordinate partition of unity and non-negativity.
- Shape function gradient consistency (finite difference vs. analytical).
- Face topology and local-to-global node mapping.
- Quadrature weight sums ($= 1/D!$ for the reference simplex).
- All-positive-weight verification for HMS and conical product rules.
- Points-inside-simplex verification for conical product.
- Polynomial exactness tests (degrees 3, 5, 7) for conical product integrators.
- `ConicalProductIntegrator<D, N>` template instantiation for $D \in \{1, 2, 3\}$, $N \in \{1, 2, 3, 4\}$.

All 49 tests and 16 element geometry tests pass as of the current revision.

6.5 File Inventory

File	Purpose
`src/geometry/SimplexCell.hh`	Reference simplex: coords, basis, topology (consteval)
`src/elements/element_geometry/SimplexElement.hh`	Element type combining geometry + integrator
`src/numerics/.../SimplexQuadrature.hh`	Classical symmetric rules (HMS, Keast)
`src/numerics/.../StroudConicalProduct.hh`	Golub–Welsch + Duffy + ConicalProductIntegrator
`src/mesh/gmsh/GmshDomainBuilder.hh`	Gmsh import with TET10 node reordering
`src/post-processing/VTK/VTKModelExporter.hh`	L2 and SPR projection, VTU export
`data/input/box_tet.geo`	Gmsh geometry for tetrahedral cantilever mesh
`data/input/box_tet10.msh`	TET10 mesh (2840 nodes)
`tests/test_simplex.cpp`	49 simplex + conical product unit tests
`main.cpp`	14-case nonlinear showcase

Chapter 7

Element Architecture: Design Patterns and Extensibility

This chapter documents the layered element architecture of `fall_n`, focusing on two central abstractions — `ElementGeometry` and `FEM_Element` — their interaction with policy-based design, and the resulting customisation points. We compare the approach against the architectures of OpenSees [11] and Kratos Multiphysics [3], highlighting trade-offs in extensibility, compile time, and runtime efficiency.

7.1 Guiding Principles

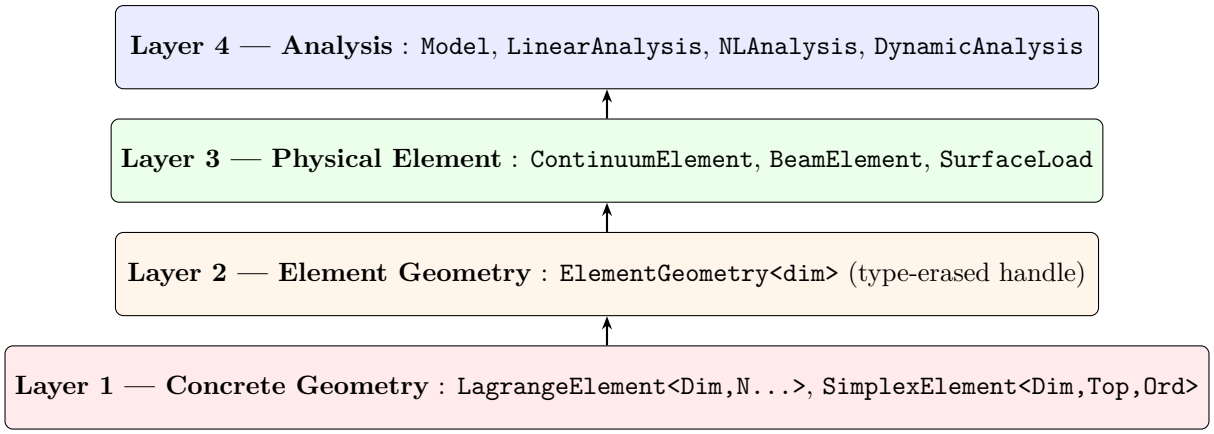
`fall_n` is designed around three principles that permeate the element subsystem:

1. **Zero-cost abstractions.** Template instantiation resolves element type, quadrature rule, kinematic formulation, and constitutive model at compile time. No virtual dispatch in the inner quadrature loop. The compiler sees the full call chain from Gauss-point integration down to the shape function evaluation, enabling inlining and SIMD vectorisation.
2. **Separation of geometry from physics.** The geometric interpolation (shape functions, Jacobian, quadrature) is decoupled from the physical element (strain–displacement operator, constitutive law, DOF assembly). The same `ElementGeometry<3>` serves both a `ContinuumElement` (3D solid) and a `SurfaceLoad` (boundary traction).
3. **Type erasure at system boundaries only.** Within a single element type, everything is monomorphic and stack-allocated. Virtual dispatch is introduced only where heterogeneity is required: storing mixed element types in a single container (`FEM_Element`) or interfacing geometry with the mesh manager (`ElementGeometry::pimpl_`).

These principles are realised through two complementary patterns: *policy-based design* (compile-time customisation) and *external polymorphism* (runtime type erasure).

7.2 Layer Decomposition

The element subsystem is organised in four layers, each with a clear responsibility boundary:



7.2.1 Layer 1: Concrete Geometry Types

These are pure value types with *no virtual functions*. They encode the reference element, shape functions, and Jacobian evaluation as compile-time-resolved operations.

LagrangeElement<Dim, N...>. A tensor-product Lagrange element parameterised by the embedding dimension `Dim` and per-axis node counts `N...` (variadic).

Instantiation	Topology	Gmsh type
`LagrangeElement<3, 2, 2, 2>`	HEX8	5
`LagrangeElement<3, 3, 3, 3>`	HEX27	12
`LagrangeElement<3, 2, 2>`	QUAD4 (surface)	3
`LagrangeElement<3, 3, 3>`	QUAD9 (surface)	10
`LagrangeElement<3, 2>`	LINE2 (edge)	1

Key design features:

- The reference cell (`geometry::cell::LagrangianCell<N...>`) is a `constexpr` object holding the nodal coordinates, the tensor-product basis, and the subentity topology — all computed at compile time.
- The topological dimension is deduced from the parameter pack: `sizeof...(N)`. Thus a surface element is simply a `LagrangeElement<3, 3, 3>` — same code path, fewer template parameters.
- Shape functions are products of 1D Lagrange polynomials, evaluated by index decomposition. No lookup table; the compiler folds constant expressions.
- The `local_index_[]` array enables node reordering (e.g., Gmsh to internal convention) without modifying the connectivity array.

SimplexElement<Dim, TopDim, Order>. A simplex element parameterised by embedding dimension, topological dimension, and polynomial order. It mirrors the `LagrangeElement` public interface but derives its reference cell from `geometry::simplex::SimplexCell<TopDim, Order>`, which stores the barycentric-based Lagrange shape functions for simplicial geometry.

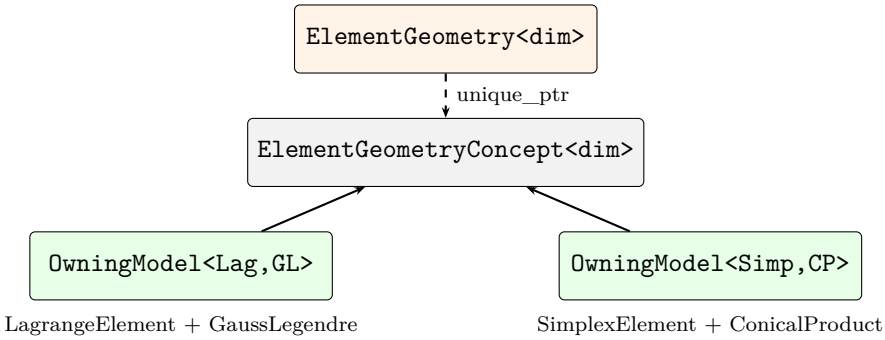
Both families share a structurally identical interface (node access, `H()`, `dH_dx()`, `evaluate_jacobian()`, `map_local_point()`) but they are *unrelated types* — there is no inheritance hierarchy linking them. Uniformity is achieved through duck-typing in the next layer.

7.2.2 Layer 2: ElementGeometry (Type-Erased Handle)

`ElementGeometry<dim>` is a type-erased value-semantic wrapper that stores *any* concrete geometry type together with its quadrature rule behind a single, uniform interface. It is the pivot of the entire element subsystem.

Pattern: External Polymorphism. The design follows the *External Polymorphism* pattern (Cleary et al. 1997, popularised by Sean Parent's "Inheritance Is the Base Class of Evil" talk):

1. **Concept** (`impl::ElementGeometryConcept<dim>`) — abstract base with pure virtual methods: `H()`, `dH_dx()`, `evaluate_jacobian()`, `differential_measure()`, `num_integration_points()` quadrature accessors, and subentity topology.
2. **Model** (`impl::OwningModel_ElementGeometry<ElementType, IntegrationStrategy>`) — template class deriving from the concept, *owning* a concrete `ElementType` and `IntegrationStrategy` by value. Each virtual override delegates to the stored objects.
3. **Handle** (`ElementGeometry<dim>`) — holds a `std::unique_ptr<ElementGeometryConcept<dim>>` (`pimpl_`). Provides value semantics via `clone()`, move defaulted.



Why External Polymorphism and not classical inheritance?

- `LagrangeElement` and `SimplexElement` remain pure value types — no vtable pointer, no heap allocation, fully `constexpr`-eligible. They can be used independently of the type-erasure layer (e.g., in constexpr unit tests).
- The virtual dispatch lives in the *wrapper*, not in the element itself. Concrete element code retains full optimisation potential.
- New element families (e.g., NURBS, serendipity, spectral) are added by writing a new value type with the required interface — no base class to inherit from, no code changes in existing elements.

Differential measure dispatch. The `differential_measure()` method in `OwningModel` uses `if constexpr` on the compile-time-known topological dimension:

```
if constexpr (topological_dim == dim)
    return std::abs(J.determinant());          // volume
else if constexpr (topological_dim == 1)
    return J.col(0).norm();                    // arc length
else if constexpr (topological_dim == 2 && dim == 3)
    return J.col(0).cross(J.col(1)).norm();    // surface area
```

Despite crossing the virtual boundary, the Jacobian matrix itself is stack-allocated (Eigen fixed-size `Matrix<double, dim, Dynamic, ..., dim, dim>`), so no heap allocation occurs inside the hot loop.

Integration via template callback. `ElementGeometry` exposes a non-virtual template method `integrate(f)` that accepts any callable `f(LocalCoordView)` and loops over quadrature points:

```
template<std::invocable<LocalCoordView> F>
auto integrate(F&& f) const {
    // ... sum w_i * |J(xi_i)| * f(xi_i) ...
}
```

Because `f` is a template parameter (not `std::function`), the compiler inlines the integrand. Only the quadrature point data (coordinates, weights) and the Jacobian evaluation pass through the virtual boundary; the user's element-level computation stays in the monomorphic world.

7.2.3 Layer 3: Physical Elements

Physical elements combine an `ElementGeometry` with material data and DOF assembly logic. Each physical element type is a class template parameterised by a *MaterialPolicy* and a *KinematicPolicy*.

ContinuumElement<MaterialPolicy, ndof, KinematicPolicy>. The workhorse for 1D/2D/3D solid mechanics:

- Holds a non-owning pointer to an `ElementGeometry<dim>` (owned by the `Domain`).
- Owns the `MaterialPoint` array (one per Gauss point), each holding a `Material<MaterialPolicy>` with its constitutive strategy.
- Provides `inject_K()`, `compute_internal_forces()`, `inject_tangent_stiffness()`, `commit_material` — the full `FiniteElement` concept interface.
- The strain–displacement operator $\mathbf{B}$ is delegated to the `KinematicPolicy` (e.g., `SmallStrain` or `TotalLagrangian`), resolved at compile time.

BeamElement<BeamPolicy, Dim, AsmPolicy>. Timoshenko beam with section integration:

- Two-node, local-frame kinematics with a rotation matrix $\mathbf{R}$.
- Integration along the beam axis uses `MaterialSection<BeamPolicy>` objects instead of `MaterialPoint`.
- The `AsmPolicy` template parameter selects between direct assembly (`DirectAssembly`) and static condensation (`CondensedAssembly`) — a compile-time strategy switch.

SurfaceLoad. Not a "physical element" in the structural sense, but uses the `ElementGeometry` quadrature machinery to compute consistent nodal forces from a distributed traction on a boundary face.

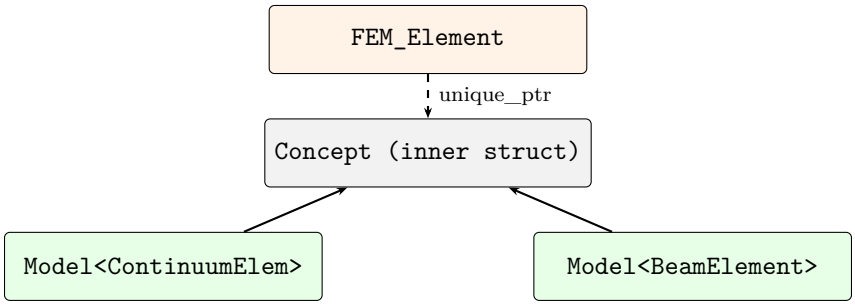
7.2.4 Layer 4: Analysis Level

`Model<MaterialPolicy, KinematicPolicy, ndofs, ElemPolicy>` owns the element container, the `Domain`, and the PETSc objects (`PetscSection`, matrices, vectors).

The `ElemPolicy` template parameter controls element storage (section 7.4.3). For homogeneous meshes, the container is a raw `std::vector<ContinuumElement<...>>` with zero overhead. For mixed meshes, it is `std::vector<FEM_Element>` with one virtual dispatch per element call.

7.3 FEM_Element: Top-Level Type Erasure

`FEM_Element` applies the same External Polymorphism pattern as `ElementGeometry`, but at the assembly level:



Key design decisions:

- **Value semantics.** Copy via `clone()`, move defaulted. No shared ownership, no reference counting.
- **Self-wrap guard.** A `requires` clause prevents wrapping a `FEM_Element` inside another `FEM_Element` (double indirection).
- **Recursive concept satisfaction.** `FEM_Element` itself satisfies the `FiniteElement` concept, verified by a `static_assert` at the bottom of the header. This means it can be nested inside other type-erased wrappers without losing the concept constraint.
- **Optional interface methods.** Mass matrix operations (`density()`, `inject_mass()`) use `if constexpr` inside `Model<T>` to detect whether the concrete element supports them, defaulting to a no-op otherwise. This avoids bloating the concept with operations not all elements provide.

There is also a `StructuralElement` wrapper, identical in pattern, that provides type safety for structural-only models (beams, shells). Both derive from *no common base class* — they are independent type-erasure handles, unified only by the `FiniteElement` concept.

7.4 Policy-Based Design: Customisation Points

`fall_n` uses policies — template parameters that select compile-time strategies — at multiple levels. Each policy is a *customisation point*: a seam where the user can inject different behaviour without modifying existing code.

7.4.1 MaterialPolicy

A *trait bag* that binds kinematic and conjugate types:

Policy	StrainT	StressT	dim
ThreeDimensionalMaterial	Strain<6>	Stress<6>	3
PlaneMaterial	Strain<3>	Stress<3>	2
UniaxialMaterial	Strain<1>	Stress<1>	1
TimoshenkoBeam3D	BeamStrain<6,3>	BeamForces<6>	3
TimoshenkoBeam2D	BeamStrain<3,2>	BeamForces<3>	2

All generic code (`Material<P>`, `MaterialPoint<P>`, `MaterialState<P>`, `ElasticRelation<P>`) is parameterised on a single `MaterialPolicy` and extracts its types via `P::StrainT`, `P::StressT`, etc.

Extensibility. Adding a shell element requires only:

1. Define `ShellMaterial<N, 3>` with appropriate `ShellGeneralizedStrain<N,3>` and `ShellResultants<N`
2. Write a constitutive model conforming to `ConstitutiveRelation<ShellMaterial<N,3»`.
3. Write the element class (`ShellElement`) satisfying `FiniteElement`.

No existing code changes; the policy mechanism propagates types automatically.

7.4.2 KinematicPolicy

Selects the strain–displacement relationship at compile time:

Policy	Description	Linear?	K_σ?
SmallStrain	$\varepsilon = \nabla^s \mathbf{u}$	yes	no
TotalLagrangian	$\mathbf{E} = \frac{1}{2}(\mathbf{F}^T\mathbf{F} - \mathbf{I})$	no	yes
UpdatedLagrangian	Cauchy strain in current config.	no	yes
Corotational	Large rotation, small strain	no	yes

Each policy is a struct with static methods:

```
struct SmallStrain {
    static constexpr bool is_geometrically_linear    = true;
    static constexpr bool needs_geometric_stiffness  = false;

    template <std::size_t dim>
    static GPKinematics<dim> evaluate(
        ElementGeometry<dim>* geo, std::size_t num_nodes,
        std::size_t ndof, const Array& Xi,
        const Eigen::VectorXd& u_e);

    template <std::size_t dim>
    static auto compute_B(ElementGeometry<dim>* geo, ...);
};
```

`ContinuumElement` calls `KinematicPolicy::evaluate<dim>()` at each quadrature point. The boolean traits steer `if constexpr` branches inside the element (e.g., add K_σ only when `needs_geometric_stiffness == true`).

Extensibility. A new formulation (e.g., logarithmic strain) is added by defining a new struct satisfying `KinematicPolicyConcept` with the required static methods and boolean traits. No changes to `ContinuumElement` or any analysis class.

7.4.3 ElementPolicy (Storage Strategy)

Controls how `Model` stores its element collection:

Policy	Container	Homogeneous	Overhead
`SingleElementPolicy<E>`	`std::vector<E>`	yes	none
`MultiElementPolicy`	`std::vector<FEM_Element>`	no	1 vtable + 1 ptr chase

The concept `ElementPolicyLike` enforces the policy interface:

```
template <typename P>
concept ElementPolicyLike = requires {
    typename P::element_type;
    typename P::container_type;
    { P::is_homogeneous } -> std::convertible_to<bool>;
} && FiniteElement<typename P::element_type>;
```

Default path. `Model<ThreeDimensionalMaterial>` defaults to: `SingleElementPolicy<ContinuumElement<ThreeDimensionalMaterial, 3, SmallStrain»`, yielding a contiguous `std::vector` of monomorphic elements — optimal for cache locality and auto-vectorisation.

7.4.4 AssemblyPolicy (Beams)

Beam elements accept an `AsmPolicy` parameter:

- `DirectAssembly` — full stiffness matrix, no condensation.
- `CondensedAssembly` — static condensation of internal DOFs before injection into PETSc.

Both are stateless tag types; the choice is resolved at compile time via `if constexpr` inside `BeamElement::inject_K()`.

7.4.5 ConstitutiveStrategy (Material Update)

The material update mechanism (`Material<P>`) delegates to a type-erased strategy object (`IntegrationStrategy<P>`), enabling runtime polymorphism over the constitutive response:

- `ElasticRelation<P>` — linear elastic.
- `HyperelasticRelation<P>` — Neo-Hookean, SVK, etc.
- `J2Plasticity<P>` — return mapping with isotropic hardening.

This is the *only* place in the element quadrature loop where runtime polymorphism is used, because the constitutive model is genuinely a runtime choice (same mesh, different materials per physical group).

7.5 Compile-Time Cost Analysis

Policy-based design with heavy template usage has a well-known compile-time cost. `fall_n` mitigates this through several strategies:

1. **Header-only by necessity, not by choice.** The library is header-only because C++ templates must be visible at the point of instantiation. However, explicit template instantiation could be added in `.cpp` files for the most common configurations to reduce redundant codegen across translation units.
2. **Type erasure as a firewall.** `ElementGeometry<dim>` and `FEM_Element` act as *template firewalls*: code that consumes an `ElementGeometry<3>` does not need to see the headers of `LagrangeElement` or `SimplexElement`. This significantly reduces transitive include graphs.
3. **Consteval reference cells.** The `LagrangianCell<N...>` and `SimplexCell<TopDim, Order>` objects with their basis functions and topological tables are `constexpr`/`consteval`, evaluated by the compiler once. They produce no runtime code unless actually called; the compiler may fold constant subexpressions entirely.
4. **Concept constraints as documentation.** The `FiniteElement`, `KinematicPolicyConcept`, `ElementPolicyLike` concepts serve as lightweight checks that produce clear error messages at instantiation time, avoiding the traditional deep-template-error-message problem.

Measured impact. A full rebuild of `fall_n` (main + 12 test targets) takes approximately 105 seconds on a single core with GCC 11 and debug optimisation (`-g`). The dominant cost is Eigen's template instantiation inside `ContinuumElement` and `BeamElement`; the element geometry layer adds minimal overhead because the type-erasure boundary limits template propagation.

7.6 Comparison with OpenSees and Kratos

7.6.1 Overview

Aspect	**OpenSees**	**Kratos**	**fall_n**
Language	C++ (C++11/14)	C++ (C++17) + Python	C++ (C++20/23)
Polymorphism model	Classical inheritance	Classical inheritance + Python bridge	Policies + External Polymorphism
Element base	`Element` (virtual)	`Element` (virtual)	`FiniteElement` (concept)
Geometry base	`CrdTransf` (virtual)	`Geometry<TPoint>` (virtual)	`ElementGeometry<dim>` (type-erased)
Material base	`UniaxialMaterial` / `NDMaterial` (virtual)	`ConstitutiveLaw` (virtual)	`Material<Policy>` (policy + type-erased strategy)
Dispatch cost	1–3 vtable calls per GP	1–2 vtable calls per GP	0 (homogeneous) or 1 (heterogeneous)

7.6.2 OpenSees

OpenSees [11] uses a deep inheritance hierarchy: `DomainComponent` → `Element` → concrete types. Materials form an even deeper tree: `Material` → `UniaxialMaterial` → specific laws.

Strengths.

- Extremely mature (20+ years), battle-tested in earthquake engineering.
- Simple runtime scripting via Tcl (later Python) — users define models without recompiling.
- Large element library (200+ elements).

Weaknesses (from an architecture perspective).

- **Monolithic virtual hierarchy.** All elements inherit from `Element`, which carries 30+ virtual methods including `getResistingForce()`, `getTangentStiff()`, etc. Adding a new element requires implementing all of them (or relying on default no-ops that may silently produce incorrect results).
- **No separation of geometry and physics.** A `Brick20N` element hard-codes its shape functions, Jacobian, AND constitutive integration in a single class. Changing the quadrature rule requires modifying the element source.
- **Dynamic dispatch in the inner loop.** Every call to `material->getStress()` and `material->getTangent()` at each Gauss point goes through virtual dispatch. While the cost is small relative to the linear algebra, it inhibits inlining and auto-vectorisation across the material boundary.
- **No concept or constraint checking.** Adding a new element with a typo in a virtual override signature compiles silently and produces incorrect results at runtime.

Contrast with fall_n. `fall_n` eliminates the monolithic base class by defining `FiniteElement` as a C++20 concept that enforces the interface at compile time. The element is free to organise its internals however it pleases; the concept checks only what the assembler needs. Geometry is factored out into `ElementGeometry`, so the same mesh can serve continuum, surface-load, and post-processing operations.

7.6.3 Kratos Multiphysics

Kratos [3] uses a more modular architecture than OpenSees, with explicit separation of geometry (`Geometry<TPoint>`), element (`Element`), condition (`Condition`), constitutive law (`ConstitutiveLaw`), and process (`Process`).

Strengths.

- **Multi-physics by design.** The `ModelPart` / `ProcessInfo` / `Variable` registry enables coupling of disparate physics (structural, fluid, thermal) sharing the same mesh.
- **Python-driven workflow.** Element selection, boundary conditions, solver parameters, and post-processing are all scriptable in Python without recompilation.
- **Geometry separated from element.** `Geometry<TPoint>` handles shape functions and Jacobian; `Element` handles physics. This is conceptually analogous to `fall_n`'s two-layer split.

Weaknesses (from an architecture perspective).

- **Runtime overhead of the variable system.** Each nodal value is stored in a hash-map-like container (`Variable<T>` keys), requiring a lookup per node per variable access. This sacrifices cache locality and prevents compile-time resolution of DOF layouts.
- **Virtual geometry hierarchy.** `Geometry<TPoint>` is an inheritance tree: `Geometry` → `Triangle2D3`, etc. Shape functions are declared `virtual`, adding dispatch cost at each Gauss point. There is no mechanism to "inline" the geometry into the element at compile time.
- **Heavyweight element interface.** `Element` carries methods for `CalculateLocalSystem`, `CalculateMassMatrix`, `CalculateDampingMatrix`, `GetDofList`, etc. Many elements implement subsets, leaving the rest as no-ops or throwing exceptions — a code-smell of interface bloat.
- **No compile-time guarantees.** Registering a new element involves adding entries to a factory map at runtime. Errors in the element's interface (wrong return type, missing method) are caught only at link time or, worse, at runtime.

Contrast with fall_n. `fall_n`'s policy-based design replaces the Kratos variable registry with compile-time type parameters: the `MaterialPolicy` selects the strain/stress types, the `KinematicPolicy` selects the B-operator, and the `ElementPolicy` selects the storage strategy — all resolved by the compiler. The trade-off is that `fall_n` requires recompilation to change element types, whereas Kratos allows runtime scripting.

7.6.4 Summary: Architectural Trade-offs

Trade-off	OpenSees	Kratos	fall_n
Runtime scripting	✓✓	✓✓✓	—
Compile-time safety	—	—	✓✓✓
Zero-cost inner loop	—	—	✓✓
Geometry–physics split	—	✓✓	✓✓✓
Heterogeneous elements	✓✓✓	✓✓✓	✓✓
Multi-physics coupling	✓	✓✓✓	—
Maturity / element count	✓✓✓	✓✓✓	✓

`fall_n` occupies a different niche: it is *not* a general-purpose production FE framework, but a research library where the primary concern is **architectural clarity and compile-time correctness** over runtime flexibility. The policy-based approach is intentionally opinionated:

it makes the "easy things trivial" (adding a new constitutive model, kinematic formulation, or element family within the existing type system) at the cost of making the "hard things hard" (mixing unrelated physics, runtime element selection from user input).

7.7 Extensibility Guide: Adding a New Element Family

This section walks through the concrete steps to add a hypothetical "Serendipity element" family, illustrating each customisation point.

1. **Layer 1: Concrete geometry type.** Create `SerendipityElement<Dim, Order>`:
   - Store node coordinates, define `H()`, `dH_dx()`, `evaluate_jacobian()`, `map_local_point()` with the serendipity basis functions.
   - No base class needed. The type must be movable and copyable.

2. **Layer 1: Integration strategy.** Create or reuse a quadrature class (`GaussLegendre2D<p>`, `ReducedIntegration<...>`, etc.) exposing `reference_integration_point(i)` and `weight(i)`.

3. **Layer 2: Instantiate through ElementGeometry.** No code changes. The constructor template:

```
ElementGeometry<3> geo(
    SerendipityElement<3, 2>{tag, node_ids},
    GaussLegendre2D<3>{});
```

   automatically instantiates an `OwningModel<Serendipity..., GL...>` and stores it behind the type-erased handle.

4. **Layer 3: Use with ContinuumElement.** No code changes. `ContinuumElement` receives a pointer to the geometry and calls `H()`, `dH_dx()`, `differential_measure()` through the virtual interface. The **B** matrix and assembly are generic.

5. **Layer 4: Mesh import.** Add a case in `GmshDomainBuilder.hh` for the Gmsh element type code, constructing `ElementGeometry` with the serendipity concrete type and the desired integrator.

6. **Optional: Type-safe model.** If the serendipity element will be used in its own model (no mixing), define:

```
using SerendipitySolid =
    ContinuumElement<ThreeDimensionalMaterial, 3,
                     SmallStrain>;
using SerendipityModel =
    Model<ThreeDimensionalMaterial, SmallStrain, 3,
          SingleElementPolicy<SerendipitySolid>>;
```

   This compiles to a contiguous vector with zero virtual dispatch.

What does NOT need to change:

- `ElementGeometry.hh` (new type is auto-detected by the templated constructor).
- `ContinuumElement.hh` (works through geometry pointer).
- `FEM_Element.hh` (type erasure is generic).
- `Model.hh` (policy-parameterised).
- Any analysis class (`LinearAnalysis`, `NLAnalysis`, etc.).
- Any material or constitutive model.

This is the central extensibility promise: **new element families are additive** — they require only new files, not modifications to existing ones — conforming to the Open/Closed Principle.

7.8 Runtime Efficiency Considerations

7.8.1 Virtual Dispatch Budget

In the homogeneous path (`SingleElementPolicy`), the element assembly loop has **zero** virtual calls for the physics: all `inject_K()`, `B()`, `H()`, `dH_dx()` calls are monomorphic and inlineable.

The only virtual calls per Gauss point are:

- `ElementGeometry::evaluate_jacobian()` — 1 vcall for the Jacobian (through pimpl).
- `ElementGeometry::H()`, `dH_dx()` — $n_{\text{nodes}} \times (1 + \text{dim})$ vcalls for shape functions and derivatives.
- `Material::compute_response()` — 1 vcall for the constitutive model (through the strategy pointer).

This is *identical per-element overhead* to OpenSees or Kratos, but `fall_n` avoids the additional vtable dispatch that those frameworks incur at the element container level.

7.8.2 Cache-Friendly Layout

`SingleElementPolicy<E>` stores all elements contiguously in a single `std::vector<E>`. Each element's `MaterialPoint` array is heap-allocated (as a `std::vector<MaterialPointT>`), but its entries are contiguous for that element.

`MultiElementPolicy` stores `FEM_Element` objects contiguously (the `unique_ptr<Concept>` wrappers are packed), but the pointed-to `Model<T>` objects are heap-allocated with potentially scattered addresses. This adds one pointer chase per element — acceptable for heterogeneous cases where the element count is moderate.

7.8.3 DOF Index Caching

`ContinuumElement` caches its flat `dof_indices_[]` array (one entry per element DOF, combining all nodes). The cache is populated lazily on first use and reused across `inject_K()`, `compute_internal_forces()`, and `inject_tangent_stiffness()`. This avoids the per-call overhead of traversing the node pointers to collect DOF indices at every assembly step.

```
void ensure_dof_cache() noexcept {
    if (dofs_cached_) return;
    collect_dof_indices();
}
```

7.8.4 Stack-Allocated Jacobians

The Jacobian matrix crossing the virtual boundary is declared as:

```
using JacobianMatrix = Eigen::Matrix<double, dim,
    Eigen::Dynamic, Eigen::ColMajor, dim, dim>;
```

The `MaxRows = dim`, `MaxCols = dim` template parameters tell Eigen to use a fixed-size stack buffer (up to $3 \times 3 = 72$ bytes) with a dynamic column count. Since `dim` ≤ 3, no heap allocation occurs, even though the column count is "dynamic" (it varies between volume and surface elements).

This ensures that the virtual call to `evaluate_jacobian()` returns by value without any `new`/`delete`, making the hot path allocation-free.

Chapter 8

Generic Boundary and Initial Condition Infrastructure

This chapter documents the design and implementation of the generic boundary condition (BC) and initial condition (IC) infrastructure for `fall_n`. The goals are:

1. **Per-DOF granularity:** constrain individual degrees of freedom (roller supports, symmetry BCs) rather than all DOFs simultaneously.
2. **Imposed (non-zero) Dirichlet:** prescribe non-zero displacement, temperature, or any nodal field value.
3. **Compile-time node selectors:** apply constraints via geometric predicates (plane, box, sphere, custom lambda) that resolve at compile time with zero runtime overhead.
4. **State transfer:** capture the complete post-solve state (`ModelState`) and use it as the initial condition for a subsequent analysis—enabling staged construction, restart, and multi-physics chaining.
5. **Extensibility:** the design should accommodate future field types (temperature, initial stress, pore pressure) without modifying the selector or constraint machinery.

8.1 Gap Analysis: the Legacy API

The original Model API exposed these methods for boundary conditions:

```
void fix_node(size_t node_idx);                          // ALL DOFs = 0
void fix_x(double val, double tol = 1e-6);               // all DOFs at x=val
void apply_node_force(size_t node_idx, auto... f);        // nodal Neumann
void apply_surface_traction(string group, auto... t);     // consistent traction
```

8.2 Design Decisions and Trade-offs

8.2.1 Compile-Time Polymorphism via Concepts

Consistent with `fall_n`'s policy-based architecture (Chapter 7), the node selector system uses C++20 concepts rather than virtual dispatch:

```
namespace fn {
template <typename S, typename NodeT>
concept NodeSelector = requires(const S& s, const NodeT& n) {
    { s(n) } -> std::convertible_to<bool>;
};
```

Table 8.1: Limitations of the legacy BC interface.

Limitation	Impact
`fix_node` constrains *all* DOFs	Cannot model roller supports, symmetry planes, or mixed BCs.
`fix_x/y/z` misnamed	These fix all three displacement components, not just the named one. Misleading to users.
No non-zero Dirichlet via Model API	`global_imposed_solution_` was created and zeroed but never populated through the Model interface. Only `DynamicAnalysis::PrescribedBC` filled it.
`ElementInitialStress` unused	The struct existed in `BoundaryCondition.hh` and was storable in `BoundaryConditionSet`, but no evaluator integrated it into the assembly pipeline.
Procedural selection	Every `fix_*` method hard-coded a coordinate loop. No reusable geometric predicate.
`VecAssemblyBegin/End` per call	`apply_node_force` triggered PETSc assembly for each node.

```
}
```

Any callable `f(const Node<dim>&)-> bool` satisfies `NodeSelector`: lambdas, function objects, and the built-in selectors all qualify. The compiler resolves the predicate at compile time, and after inlining a `PlaneSelector` collapses to a single `fabs` + comparison—zero overhead compared to a hand-written loop.

Why not `std::function`? A `std::function<bool(const Node&)>` would erase the type and force heap allocation for lambdas that exceed the small-buffer size (typically 16–32 bytes). For selectors applied to every node in the mesh, this per-call overhead is measurable. The concept-based approach pays nothing at runtime.

Why not virtual? A virtual `AbstractSelector::operator()` would add one indirect call per node per constraint pass. With 10 000 nodes this is $\sim$10 μs (branch-predictor penalty), acceptable but unnecessary when the concrete type is known at compile time.

8.2.2 Constraint Storage: Extending the Existing Map

The existing `ConstraintDofInfo` type already maps sieve point IDs to (DOF index list, prescribed value list):

```
using ConstraintDofInfo =
  std::map<PetscInt,
           std::pair<std::vector<PetscInt>,
                     std::vector<PetscScalar>>>;
```

The new API *extends* this map rather than replacing it. The key operations are:

1. **Insert in sorted order.** PETSc's `PetscSectionSetConstraintIndices` requires increasing DOF indices. The `constrain_dof` method uses `std::lower_bound` to maintain this invariant.
2. **Merge, not overwrite.** If DOF 0 of a node is already constrained and the user constrains DOF 2, both are stored. An update to DOF 0 changes the prescribed value without duplicating the index.

8.2.3 Non-Zero Dirichlet: Where Values Flow

Non-zero prescribed values are stored in `constraints_` alongside the DOF indices. During `setup()`, a new `fill_imposed_solution()` step extracts these values from the constraint map into the PETSc local vector `global_imposed_solution_`:

```
void setup() {
    setup_boundary_conditions();   // PETSc section
    setup_vectors();               // create Vecs
    fill_imposed_solution();       // NEW: u_D -> imposed
    setup_matrix();                // create Mat
}
```

Remark 8.1. Non-zero Dirichlet requires that the analysis accounts for the coupling $\mathbf{K}_{fc}\,\mathbf{u}_c$ in the right-hand side. `NonlinearAnalysis` handles this naturally: the internal forces $\mathbf{f}_{\text{int}}(\mathbf{u}_{\text{free}} + \mathbf{u}_D)$ include the prescribed part. `LinearAnalysis` currently does *not* apply this correction—for non-zero Dirichlet, users should solve with `NonlinearAnalysis` (which converges in one Newton step for linear problems).

8.2.4 ModelState: Platform-Independent Snapshot

For state transfer between models, we need a data structure that:

- Does *not* depend on PETSc (no `Vec`, `Mat`).
- Does *not* depend on the `MaterialPolicy` template parameter (so an elastic model's state can seed a plastic model).
- Is copyable, movable, and (future) serialisable.

`ModelState<dim>` satisfies all three. It stores:

- Per-node: ID, displacement, velocity.
- Per-element, per-Gauss-point: stress and strain in Voigt notation (`std::array<double, nvoigt>`).

The PETSc-to-array extraction in `capture_state()` uses `VecGetArrayRead` (zero-copy read) and the element's `material_points()` accessor which recomputes stress from the committed strain via the constitutive relation.

8.3 Implementation

8.3.1 NodeSelector.hh — Built-in Selectors

Table 8.2: Built-in node selectors.

Selector	Predicate	Parameters
`PlaneSelector<dim>`	$\lvert x_i - v\rvert < \varepsilon$	axis i, value v, tolerance ε
`BoxSelector<dim>`	$x \in [\text{lo}, \text{hi}]$	corners, tolerance
`SphereSelector<dim>`	$\lVert x - c\rVert \le r$	centre c, radius r, tolerance
`NodeIdSelector`	$\text{id} \in S$	set of node IDs

Combinators turn simple selectors into compound predicates:

```
auto corner = fn::select_and(
    fn::PlaneSelector<3>{0, 0.0},    // x = 0
    fn::PlaneSelector<3>{1, 0.0}     // y = 0
);
auto not_x0 = fn::select_not(fn::PlaneSelector<3>{0, 0.0});
auto union_ = fn::select_or(selectorA, selectorB);
```

Convenience factories (`fn::on_plane`, `fn::in_box`, `fn::in_sphere`, `fn::node_ids`) provide a terser syntax.

8.3.2 Model.hh — New Constraint Methods

Table 8.3: New Model constraint API.

Method	Description
`constrain_dof(n, d, v)`	Constrain DOF d of node n to value v. Sorted insertion.
`constrain_node(n, vals)`	Constrain all DOFs of node n to the given array of values.
`constrain_dof_where(sel, d, v)`	Constrain DOF d at every node matching selector `sel`.
`constrain_where(sel)`	Constrain all DOFs (homogeneous) at every matching node.
`constrain_where(sel, vals)`	Constrain all DOFs with prescribed values at every matching node.
`is_constrained(n)`	Query: is any DOF of node n constrained?
`is_dof_constrained(n, d)`	Query: is DOF d of node n constrained?
`prescribed_value(n, d)`	Get the prescribed displacement at DOF d (0 if free).
`num_constraints()`	Total number of constrained DOFs across all nodes.
`capture_state()`	Extract current solution into a `ModelState<dim>`.
`apply_initial_displacement(s)`	Populate imposed solution from a `ModelState<dim>`.

8.3.3 Backward Compatibility

The legacy methods `fix_node`, `fix_x`, `fix_y`, `fix_z`, and `fix_orthogonal_plane` remain unchanged in semantics—they continue to constrain *all* DOFs of every matching node. Internally, `fix_orthogonal_plane` now delegates to:

```
void fix_orthogonal_plane(int i, double val, double tol) {
    constrain_where(fn::PlaneSelector<dim>{i, val, tol});
}
```

The code path is identical; only the implementation reuses the selector infrastructure.

8.4 Use-Case Walkthrough: Roller vs. Clamped Cantilever

Consider a cantilever beam $[0, L]\times[0, b]\times[0, h]$ loaded at $x = L$ by a surface traction $\mathbf{t} = (0, t_y, t_z)$.

Clamped support (legacy):

```
M.fix_x(0.0);    // constrains u_x, u_y, u_z at x=0
```

Equivalent new API:

```
M.constrain_where(fn::PlaneSelector<3>{0, 0.0});
```

Roller support (new API): Only $u_x = 0$ at $x = 0$. Rigid body prevention requires additionally constraining u_y and u_z at isolated points:

```
// u_x = 0 on entire face
M.constrain_dof_where(fn::PlaneSelector<3>{0, 0.0}, 0, 0.0);

// Prevent rigid translation + rotation about x:
auto origin = fn::select_and(fn::PlaneSelector<3>{0, 0.0},
    fn::select_and(fn::PlaneSelector<3>{1, 0.0},
                   fn::PlaneSelector<3>{2, 0.0}));
M.constrain_dof_where(origin, 1, 0.0);  // u_y at origin
M.constrain_dof_where(origin, 2, 0.0);  // u_z at origin

auto top_z = fn::select_and(fn::PlaneSelector<3>{0, 0.0},
    fn::select_and(fn::PlaneSelector<3>{1, 0.0},
                   fn::PlaneSelector<3>{2, h}));
M.constrain_dof_where(top_z, 1, 0.0);   // prevents Rx
```

The roller case has 30 constrained DOFs (28 nodes $\times$ 1 + 2 extra) versus 84 (28 $\times$ 3) for the clamped case. With fewer constraints, the beam is more flexible:

Table 8.4: Roller vs. clamped comparison (HEX8 beam, $E = 200$, $\nu = 0.3$).

BC type	**Constrained DOFs**	$\max\lvert u\rvert$	**Ratio**
Clamped	84	0.594	1.00
Roller (u_x only)	30	2.762	4.64×

This confirms: the per-DOF mechanism works correctly and produces a physically meaningful stiffer/more-flexible response.

8.5 State Capture and Continuation Analysis

8.5.1 ModelState Structure

```
template <size_t dim>
struct ModelState {
    static constexpr size_t nvoigt = dim * (dim + 1) / 2;

    struct NodalData {
        size_t node_id;
        array<double, dim> displacement;
        array<double, dim> velocity;        // zero for static
    };

    struct GaussPointData {
        array<double, nvoigt> stress;
        array<double, nvoigt> strain;
    };

    struct ElementData {
        size_t element_index;
        vector<GaussPointData> gauss_points;
    };

    vector<NodalData>   nodes;
    vector<ElementData> elements;
```

```
};
```

8.5.2 Capture Workflow

`Model::capture_state()` extracts:

1. **Nodal displacements:** from the PETSc local vector `current_state_` via `VecGetArrayRead` (zero-copy). Each node's DOF offsets from `node.dof_index()` are used to index into the raw array.
2. **Element stress/strain:** from each `MaterialPoint`'s committed strain (`current_state()`) and recomputed stress (`compute_response()`). The Voigt vectors are copied component-by-component into `std::array<double, nvoigt>`, guarded by the compile-time `num_components` constant.

8.5.3 Future: Initial Stress Integration

The existing `ElementInitialStress<dim>` struct in `BoundaryCondition.hh` can be populated from `ModelState::ElementData`:

```
for (const auto& ed : state.elements) {
    ElementInitialStress<dim> eis;
    eis.element_index = ed.element_index;
    for (const auto& gp : ed.gauss_points) {
        eis.gauss_point_stresses.push_back(gp.stress);
    }
    bcs.add_initial_stress(std::move(eis));
}
```

For this to take effect, the element assembly must incorporate $\boldsymbol{\sigma}_0$ into the internal force:

$$\mathbf{f}_{\text{int}}(\mathbf{u}) = \int_{\Omega_e} \mathbf{B}^T \left(\boldsymbol{\sigma}(\boldsymbol{\varepsilon}(\mathbf{u})) + \boldsymbol{\sigma}_0\right) \mathrm{d}V$$

This requires modifications to `ContinuumElement::compute_internal_force_vector`, specifically adding a σ`_initial` term at each Gauss point. This integration is deferred to a future phase.

8.6 Comparison with OpenSees and Kratos

Table 8.5: BC/IC handling comparison across frameworks.

Feature	OpenSees	Kratos	fall_n
Single-DOF fix	`fix nodeTag dof`	`DISPLACEMENT_X: 0.0`	`constrain_dof(n,d,v)`
Geometric selection	Tcl loops	Python `FindNodalH`	`PlaneSelector`, lambda
Non-zero Dirichlet	`imposedMotion` pattern	`ApplyDirichletProcess`	`constrain_dof_where(s,d,v)`
State transfer	`-database` serialise	`ModelPartIO`	`ModelState` capture/apply
Initial stress	Per-element update	`INITIAL_STRESS_TENSOR`	Defined, not yet integrated
Dispatch cost	Virtual (runtime)	Virtual + Python (runtime)	Compile-time (zero)

Key differences:

- **OpenSees** uses a tag-based Tcl scripting layer for BC specification. Geometric loops require user scripting. State transfer uses a persistence layer (`FE_Datastore`).
- **Kratos** uses Python processes (`ApplyDirichletProcess`, `ApplyNeumannProcess`) that iterate over model parts. Highly flexible but with Python → C++ boundary overhead.
- **fall_n** resolves all selector logic at compile time. The constraint map lives in pure C++ with PETSc-section-based enforcement. No scripting layer, no virtual dispatch per node.

8.7 Extensibility Roadmap

The current infrastructure is designed for forward extension. Table 8.6 outlines planned capabilities and their implementation path.

Table 8.6: Extensibility roadmap.

Capability	Implementation path	Status
Per-DOF constraints	`constrain_dof`, `constrain_dof_where`	**Implemented**
Non-zero Dirichlet	`fill_imposed_solution()` + NL correction	**Implemented** (NL only)
Node selectors	`NodeSelector` concept + built-in selectors	**Implemented**
State capture/transfer	`ModelState` + `capture_state`	**Implemented**
Initial stress	Extend `ContinuumElement::compute_internal_force` to add $\boldsymbol{\sigma}_0$ at each GP	Designed, deferred
Body forces	Consistent nodal load via $\int N^T \mathbf{b}\,\mathrm{d}V$	Planned
Temperature field	Additional `ModelState` field + thermal strain $\boldsymbol{\varepsilon}_\theta = \alpha\,\Delta T\,\mathbf{1}$	Planned
Multi-step builder	Fluent builder with chained `.constrain().material().build()`	Planned
State serialisation	HDF5/JSON export of `ModelState`	Planned
Linear non-zero BC	RHS correction $\mathbf{f}_{\text{eff}} = \mathbf{f}_{\text{ext}} - \mathbf{K}_{fc}\,\mathbf{u}_D$ in `LinearAnalysis`	Planned

8.8 Test Suite

The implementation is validated by 81 tests in `tests/test_boundary_conditions.cpp`, organized as follows:

Table 8.7: Test suite structure (81 tests).

Category	Count	Coverage
Node selectors (pure geometry)	38	PlaneSelector, BoxSelector, SphereSelector, NodeIdSelector, combinators, la
ModelState data structure	9	Default state, add/query, copy semantics
Per-DOF constraints	11	Single DOF, multiple DOFs, non-zero value, value update
Selector-based constraints	10	PlaneSelector on DOF, all DOFs, values, lambda, composite
Full solve: roller vs. clamped	3	Clamped > 0, Roller > 0, Roller > Clamped
State capture after solve	7	Non-empty, node count, element count, non-zero displacement, non-zero str
Backward compatibility	3	`fix_x` gives positive deflection, reasonable magnitude

8.9 Summary

The generic BC/IC infrastructure extends `fall_n` from an all-or-nothing clamped-support tool to a framework capable of:

- **Arbitrary Dirichlet BCs:** per-DOF, per-node, per-selector, with prescribed values (zero or non-zero).
- **Zero-cost selectors:** compile-time geometric predicates with no runtime overhead versus hand-written loops.
- **State snapshots:** PETSc-independent `ModelState` for continuation, restart, and future serialisation.
- **Full backward compatibility:** `fix_x`, `fix_y`, `fix_z` continue to work exactly as before.

The design follows the same policy-based, compile-time polymorphic philosophy established in the element architecture (Chapter 7) and validated by an 81-test suite.

Chapter 9

ElementGeometry Refactor

This chapter documents the refactor of the `ElementGeometry` subsystem. The goal of the change was not to redesign the whole geometry layer from scratch, but to fix the most important semantic and architectural weaknesses without degrading the hot path of element integration.

The guiding constraint was explicit: *prefer compile-time cost over runtime cost whenever the trade-off is reasonable.* In other words, stronger type constraints, cleaner ownership, and slightly more template work at compile time are acceptable, whereas extra runtime indirection inside the Gauss loop is not.

9.1 Problem Statement

Before the refactor, `ElementGeometry<dim>` already had a good high-level idea: concrete geometry and quadrature strategy were stored behind a type-erased, value-semantic wrapper. However, three problems were architecturally significant.

9.1.1 Broken value semantics

The wrapper cloned only its internal `pimpl_`, while metadata such as `integration_point_`, `sieve_id`, and `physical_group_` lived outside the cloned state. As a consequence, copy construction and copy assignment were semantically incomplete: two `ElementGeometry` objects could share the same geometric map while disagreeing on already-materialised Gauss-point data or mesh metadata.

This violated a core expectation of value types:

$$\texttt{b = a} \quad \Longrightarrow \quad \texttt{b} \text{ must represent the same logical value as } \texttt{a}.$$

9.1.2 Integration strategy mixed into concrete geometry headers

The concrete geometry headers contained both the element type and the quadrature strategy implementation. This worked, but it blurred the boundary between two distinct concepts:

- a geometric interpolation on a reference cell, and
- a numerical integration policy on that reference cell.

Architecturally this was undesirable because the element family and the quadrature family evolve for different reasons.

9.1.3 Weak compile-time invariants

Some invalid states were still representable:

- unsupported Gauss–Legendre orders silently produced zero-filled arrays;
- `detJ()` was visible even for embedded geometries with non-square Jacobians;

- tensor-product geometries did not consistently enforce `topological_dim` $\leq$ `dim`.

These were not merely cosmetic issues. They weakened the library contract and made failures harder to interpret.

9.2 Design Goals

The refactor was driven by the following goals:

1. Preserve the existing *external polymorphism* structure.
2. Keep quadrature evaluation monomorphic and inlineable inside the concrete model.
3. Repair value semantics of the wrapper.
4. Make invalid geometric or quadrature configurations fail early.
5. Reduce conceptual coupling between geometry and integration.

Equally important is what was *not* attempted in this refactor: the geometry layer still depends on `Node<dim>`, which means it is not yet a purely geometric vertex layer. That larger ontological cleanup remains a future step because it cuts across `Model`, DOF storage, and multiple element families.

9.3 Refactor Overview

9.3.1 Metadata moved under wrapper ownership

`ElementGeometry` now owns an internal metadata aggregate containing:

- the materialised integration points,
- the optional DMPlex sieve identifier, and
- the physical-group label imported from Gmsh.

This has two consequences.

First, copy construction and copy assignment now copy both the cloned geometry implementation and the metadata, restoring proper value semantics.

Second, the metadata is no longer a public bag of fields. It is accessed through a narrow API: `sieve_id()`, `has_sieve_id()`, `physical_group()`, `integration_points()`, etc.

This is a direct SRP improvement: the wrapper remains responsible for its own logical state, and downstream code no longer reaches into its internals.

9.3.2 Quadrature strategies extracted to dedicated headers

Two strategy types were extracted:

- `GaussLegendreCellIntegrator.hh`
- `SimplexIntegrator.hh`

The concrete geometry classes remain pure descriptions of interpolation and mapping. The numerical integration policy is still attached at compile time, but it is no longer embedded in the same header for incidental reasons.

This separation is conceptually important. In isoparametric finite elements, geometry defines the map

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{a=1}^{n_n} N_a(\boldsymbol{\xi})\, \mathbf{x}_a,$$

while quadrature defines an approximation of

$$\int_{\hat{\Omega}} f(\boldsymbol{\xi})\, \mathrm{d}\hat{\Omega} \;\approx\; \sum_{g=1}^{n_g} w_g\, f(\boldsymbol{\xi}_g).$$

They are related in use, but they are not the same abstraction.

9.3.3 A small quadrature concept

The new `QuadratureStrategy` concept formalises the contract expected by `ElementGeometry`'s internal model:

- `num_integration_points`
- `reference_integration_point(i)`
- `weight(i)`

The benefit is not runtime polymorphism. The benefit is that failures move to compile time, where the error is cheaper and more local.

9.4 Geometry and Measure

The refactor keeps the existing zero-cost logic for the differential measure. Given the Jacobian $J = \partial\mathbf{x}/\partial\boldsymbol{\xi}$, the integration measure depends on the relationship between topological and embedding dimensions:

$$\text{full-dimensional cell:} \quad \mathrm{d}\Omega = |\det J|\,\mathrm{d}\hat{\Omega}, \tag{9.1}$$

$$\text{embedded curve:} \quad \mathrm{d}s = \|J\|\,\mathrm{d}\hat{s}, \tag{9.2}$$

$$\text{embedded surface in } \mathbb{R}^3: \quad \mathrm{d}S = \|J_1 \times J_2\|\,\mathrm{d}\hat{S}. \tag{9.3}$$

This dispatch still happens inside `OwningModel_ElementGeometry<ElementType, IntegrationStrategy>` using `if constexpr`. Therefore the compiler sees the concrete element type, the concrete topological dimension, and the concrete matrix shape. No extra runtime branch was introduced inside the integration loop.

9.5 API Hardening

9.5.1 Restricting `detJ()`

`detJ()` now exists only when the Jacobian is square, namely when the element is full-dimensional:

$$\texttt{topological_dim} = \texttt{dim}.$$

This is important because a determinant is not defined for the Jacobian of an embedded beam or surface. Exposing the member unconditionally was misleading: it suggested a mathematically meaningful operation where none exists.

9.5.2 Fail-fast Gauss–Legendre tables

Unsupported Gauss–Legendre orders no longer fall back to zero arrays. Instead, they trigger a compile-time error. This is the right contract for a template-based numerical kernel: if the requested order is unsupported, the program should fail where the invalid instantiation occurs.

9.5.3 Safer tensor-product constraints

`LagrangeElement<Dim, N...>` now enforces that the number of natural coordinates does not exceed the embedding dimension. This prevents invalid type combinations from entering the system.

9.6 Materialised Integration Points

The wrapper continues to precompute physical Gauss-point coordinates when the domain requests them. The refactor extends this step so that the integration weight is also stored in each materialised point.

This is useful for post-processing and point-wise objects attached to Gauss points because the point now carries a coherent local state:

- physical coordinates,
- global identifier for VTK export,
- quadrature weight.

The operation remains amortised and explicit. It does not change the quadrature path used in stiffness or residual evaluation.

9.7 Performance Discussion

The refactor was intentionally biased toward preserving runtime behaviour.

9.7.1 What did not change

- Concrete geometry evaluation is still template-based.
- The Jacobian and shape-function evaluation remain statically resolved.
- The quadrature loop still uses direct calls on the concrete strategy stored inside the model.
- No new virtual call was inserted inside `integrate()`.

9.7.2 What did change

- More errors are now diagnosed at compile time.
- Integration strategies live in dedicated headers, reducing conceptual coupling and making future dependency management easier.
- Copying `ElementGeometry` is slightly more expensive if integration points have already been materialised, because the copy now correctly copies them. This is not a regression; it is the real cost of correct value semantics.

This is the intended trade-off. Copying a fully materialised geometric entity is not a hot-path operation in FEM assembly, whereas incorrect copies are architecturally dangerous.

9.8 What Was Deliberately Deferred

The refactor does *not* yet solve the deeper issue that the geometry layer depends on `Node<dim>`, and `Node<dim>` already contains DOF state. From a strict modelling standpoint, a geometric layer should ideally depend on a lighter vertex concept carrying only:

- identifier,
- coordinates,
- optional mesh-backend handle.

That separation would improve the ontology of the codebase and further align the design with SRP and DIP. However, it affects much more than `ElementGeometry`; it is therefore better handled as a dedicated follow-up refactor instead of being folded into the present change.

9.9 Result

Architecturally, the refactor moves `ElementGeometry` from “good idea with leaky semantics” toward “stable boundary abstraction”.

The most important outcomes are:

1. wrapper copies are now semantically correct;
2. geometry and quadrature are more cleanly separated;
3. invalid numerical configurations fail early;
4. the hot numerical path remains compile-time resolved.

This is a meaningful improvement even though the subsystem is not yet the final form of the geometry layer. It removes a real correctness bug, strengthens the public contract, and prepares the code for a later, larger cleanup around geometric vertices versus analysis nodes.

Chapter 10

Serendipity Element Families

This chapter documents the implementation of the new serendipity element families added to `fall_n`, their integration into `ElementGeometry`, the associated tests, and the benchmark results.

The implementation goal was architectural, not merely additive: to introduce a new family without changing the overall layering of the codebase. In particular:

- the concrete element remains a pure value type;
- `ElementGeometry` continues to provide the type-erased boundary;
- quadrature remains a separate compile-time policy;
- no new virtual dispatch is introduced in the hot integration path.

10.1 Why Serendipity?

For tensor-product cells, the quadratic Lagrange family and the quadratic serendipity family differ substantially in node count:

Reference cell	Quadratic Lagrange	Quadratic serendipity
Quadrilateral	9 nodes	8 nodes
Hexahedron	27 nodes	20 nodes

The serendipity family removes face-center and volume-center nodes while retaining vertex and edge nodes. This makes the element attractive when the primary objective is to reduce geometry, interpolation, and assembly cost without dropping to first order.

Architecturally, serendipity is also a good stress test of the current design. It requires:

- a new reference cell type;
- a new concrete geometry type;
- VTK support for new node counts;
- optional Gmsh import support;
- no changes in the physical element layer.

That is precisely the extensibility story claimed in section 7.7; this chapter documents the claim becoming real code.

10.2 Mathematical Structure

10.2.1 Tensor-product serendipity on the cube

For the quadratic serendipity family on the reference hypercube $\hat{\Omega} = [-1, 1]^d$, nodes are located at:

- the 2^d vertices;

- the $d\,2^{d-1}$ edge midpoints.

Thus, for $d = 3$, the quadratic hexahedron has

$$2^3 + 3 \cdot 2^2 = 8 + 12 = 20$$

nodes.

Corner shape functions. Let $\mathbf{r} = (r_1, \ldots, r_d)$ be a corner node with $r_j \in \{-1, 1\}$. The quadratic serendipity shape function is

$$N_{\mathbf{r}}(\mathbf{x}) = 2^{-d}\left(\prod_{j=1}^{d}(1 + r_j x_j)\right)\left(1 + \sum_{j=1}^{d} r_j x_j - d\right).$$

Edge-midpoint shape functions. Let $\mathbf{r}$ be an edge node, so exactly one component $r_k = 0$ and all others are ± 1. Then

$$N_{\mathbf{r}}(\mathbf{x}) = 2^{1-d}(1 - x_k^2) \prod_{j \neq k}(1 + r_j x_j).$$

These expressions unify the 1D, 2D, and 3D quadratic serendipity family. The implementation therefore uses a single generic formula for `TopDim` $= 1, 2, 3$, with compile-time node descriptors and `consteval` reference-node tables.

10.2.2 Why the tetrahedral "serendipity" family is different

The simplex case requires more care. For tetrahedra at order $p = 2$, the standard quadratic Lagrange simplex already uses only:

- 4 vertices;
- 6 edge midpoints;
- no face-interior nodes;
- no volume-interior nodes.

Therefore the practical quadratic tetrahedral family is already "serendipity-like" in the sense relevant to this codebase: there are no face-center or body-center DOFs to remove.

For this reason, the new `SerendipitySimplexElement` is represented as an explicit family alias over the existing simplex implementation for orders 1 and 2. This is mathematically honest and avoids inventing an ad hoc reduced tetrahedron with unclear approximation properties.

10.3 Implementation Structure

10.3.1 Reference cells

The tensor-product family is implemented in `SerendipityCell.hh`. The file introduces:

- `serendipity_num_nodes(top_dim, order)`;
- `serendipity_reference_nodes<TopDim, Order>()`;
- `SerendipityBasis<TopDim, Order>`;
- `SerendipityCell<TopDim, Order>`.

Reference nodes are computed with `consteval`, so the full node layout is available at compile time and can participate in static assertions and compile-time topology generation.

10.3.2 Concrete geometry

The concrete tensor-product family is implemented in `SerendipityElement.hh` as

`SerendipityElement<Dim, TopDim, Order>`,

mirroring the shape of the existing `SimplexElement` and `LagrangeElement` abstractions:

- it stores node IDs and bound node pointers;
- it exposes `H()`, `dH_dx()`, `map_local_point()`, and `evaluate_jacobian()`;
- it carries a `ReferenceCell` type with compile-time topology.

The new family is therefore a first-class Layer 1 geometry type.

10.3.3 ElementGeometry integration

`ElementGeometry.hh` was extended in only one place: its face-topology and face-factory logic now recognises the serendipity family. No changes were required in the outer wrapper API.

This confirms the architectural objective: the extension point is the concrete geometry type, not the type-erasure layer.

10.3.4 Quadrature choices

The following quadratures were selected from the rules already present in the library:

Element	Quadrature	Rationale
HEX20	`GaussLegendreCellIntegrator<3,3,3>`	full quadratic tensor-product integration
QUAD8 face	`GaussLegendreCellIntegrator<3,3>`	compatible face integration
EDGE3 face	`GaussLegendreCellIntegrator<3>`	compatible edge integration
TET10 serendipity	`SimplexIntegrator<3,2>`	same requirements as quadratic simplex

The choice is intentionally conservative. The goal of this first integration is correctness and compatibility, not aggressive reduced integration.

10.4 VTK and Gmsh Support

10.4.1 VTK

`VTKCellTraits.hh` now recognises:

- 8-node quadratic serendipity quadrilaterals (`VTK_QUADRATIC_QUAD`);
- 20-node quadratic serendipity hexahedra (`VTK_QUADRATIC_HEXAHEDRON`).

The test executable writes four validation files:

- `data/output/serendipity_hex20_sample_mesh.vtu`
- `data/output/serendipity_hex20_sample_gauss.vtu`
- `data/output/serendipity_tet10_sample_mesh.vtu`
- `data/output/serendipity_tet10_sample_gauss.vtu`

These contain both the cell geometry and the materialised Gauss cloud, together with a simple scalar field for visual inspection in ParaView.

10.4.2 Gmsh

`GmshDomainBuilder.hh` was extended with support for:

- `HEX_20` (element type 17);
- `QUA_8` (element type 16) for boundary faces.

The chosen internal node ordering matches the standard corner-plus-edge layout, so no non-trivial permutation was required for these two element types.

10.5 Validation Tests

The new test executable `test_serendipity.cpp` validates:

1. static node counts for the family;
2. reference-node coordinates for HEX20;
3. partition of unity of the HEX20 basis;
4. Kronecker interpolation property at all 20 reference nodes;
5. exact reproduction of a representative quadratic polynomial;
6. unit-cube volume and mixed-monomial integration through `ElementGeometry`;
7. creation and integration of quadratic serendipity face geometry;
8. partition of unity and volume for the tetrahedral serendipity family;
9. VTK cell-type dispatch and actual file generation.

This test set is deliberately front-loaded toward interpolation quality. Before benchmarking or embedding the element into larger analyses, the basis itself must satisfy the core finite element identities.

10.6 Benchmark Results

The microbenchmark measures repeated geometric integration of a constant field through `ElementGeometry`. It was executed locally after the full test suite passed.

Case	Wall time [s]	Observation
HEX20 serendipity	6.04	baseline for 4000 integrations
HEX27 triquadratic	19.14	about 3.2× slower than HEX20
TET10 serendipity(simplex)	0.334	practically same as quadratic simplex
TET10 quadratic simplex	0.364	expected baseline

Interpretation. The HEX20 result is the important one. The node reduction from 27 to 20 lowers the cost of geometry interpolation and Jacobian assembly enough to be visible even though both elements use the same $3 \times 3 \times 3$ Gauss rule.

The tetrahedral result confirms the modelling claim made earlier: at quadratic order, "serendipity tetrahedron" and the existing quadratic simplex are effectively the same finite element family in this codebase.

Iteration after benchmarking. No further optimisation was required after the first measurement because the results already matched the intended architectural outcome:

- reduced tensor-product node count translated into lower runtime cost;
- simplex serendipity remained mathematically and computationally aligned with the existing implementation;
- no additional runtime indirection was introduced.

10.7 Architectural Takeaways

This extension validates several design choices of the current library:

1. **External polymorphism works.** A new family was added without redesigning `ElementGeometry`.
2. **Reference-cell-centric design scales.** Static reference nodes and subentity topology remain the right place to encode geometric knowledge.
3. **Geometry and physics remain decoupled.** The new elements plug into the same generic physical element and VTK machinery.

4. **The simplex/tensor-product split is conceptually sound.** The tetrahedral case did not need an artificial reduced basis, while the hexahedral case benefited from a genuinely different family.

10.8 Future Work

The present implementation intentionally stops at order 2. Sensible next steps would be:

- reduced-integration variants for HEX20;
- serendipity quadrilateral usage in shell formulations;
- higher-order incomplete tensor-product families if they are needed;
- a more explicit distinction in the type system between "quadratic simplex" and "serendipity simplex" if higher-order simplex variants are later introduced.

At this stage, however, the library now has a complete and usable quadratic serendipity pathway for both the tensor-product and simplex branches, with tests, benchmark coverage, and VTK output.

Chapter 11

Domain Refactor Ontology

This chapter does not yet present the final implementation of the `Domain` refactor. Its purpose is more foundational: to fix the *ontology* of the codebase before more invasive changes are made.

The current implementation already contains several useful abstractions: `Point`, `Node`, `IntegrationPoint`, `MaterialPoint`, `NodeSection`, `MaterialSection`, and `SectionGeometry`. The architectural difficulty is that some of these types represent *different concepts that happen to coincide geometrically*, while others represent *the same physical location under different numerical roles*. If those roles are not separated explicitly, `Domain` becomes the place where geometry, analysis state, quadrature, and constitutive bookkeeping collapse into a single layer.

The refactor should therefore begin by answering a simple question:

> What kinds of "points" exist in the library, and what exactly does each one mean?

11.1 Architectural Diagnosis

At present, the library contains three partially overlapping strata:

1. **Geometric entities:** reference cells, mapped element geometry, coordinates, frames, section shapes.
2. **Discrete unknown carriers:** nodal degrees of freedom, constrained DOFs, PETSc indexing.
3. **Sampling / constitutive sites:** Gauss points, material points, beam sections, shell sections.

The main architectural leak is that `Node` currently belongs to both the first and second strata at the same time. It is a geometric point, but it is also the owner of nodal DOF indices and PETSc-related mesh indexing. This is convenient for a first implementation, but it prevents a clean `Domain` abstraction because a geometry-only domain should not need to know anything about nodal unknown layouts.

There is a second leak in the repeated use of the word *section*. In the current code, *section* may refer to:

- a structural cross-section shape (`SectionGeometry`);
- a section plane located at a node (`NodeSection`);
- a constitutive section sample along a beam or shell (`MaterialSection`);
- a PETSc `PetscSection`, which is purely an indexing map and not a structural section at all.

These are not small naming issues. They correspond to genuinely distinct mathematical objects.

11.2 Entity-by-Entity Semantics

11.2.1 `Point<Dim>`

`Point<Dim>` is the cleanest entity in the current design. It is a coordinate value type: a point in physical space with no identity, no topology, no DOFs, no material, and no ownership semantics beyond ordinary C++ value semantics.

What it is. A metric object in $\mathbb{R}^{\mathrm{Dim}}$.

What it is not. It is not a mesh vertex, not a quadrature point, not a node in the FEM sense, and not a constitutive point.

Architectural role. `Point` should remain the lowest-level geometric atom. Any richer entity that has coordinates should refine or contain a `Point`.

11.2.2 `Vertex<Dim>` (proposed and introduced in this step)

`Vertex<Dim>` is the missing geometry/topology abstraction. It is a `Point` with a stable mesh identity, but without analysis state. In other words:

$$\texttt{Vertex} = \texttt{Point} + \texttt{mesh identity}.$$

What it is. A vertex of the discrete domain mesh. It represents a topological 0-cell and its physical coordinates.

What it is not. It is not a carrier of primary unknowns. It has no nodal DOFs, no constraint state, and no constitutive variables.

Why it matters. If `Domain` is supposed to live at the geometric level, then it should own `Vertex` objects, not `Node` objects. The new `Vertex.hh` introduced in this iteration fixes the intended target abstraction without forcing an immediate rewrite of every consumer.

11.2.3 `Node<Dim, Storage>`

`Node` is currently defined as

$$\texttt{Node} = \texttt{Point} + \texttt{id} + \texttt{DoF storage} + \texttt{PETSc/DMPlex indexing metadata}.$$

What it is. An analysis-layer discrete unknown carrier. More precisely, it is a nodal site where one or more primary fields are stored and indexed globally.

What it is not. It should not be interpreted as the pure geometric vertex of the domain.

Current status. The current implementation uses `Node` in both senses: as the thing owned by the domain, and as the thing that carries DOFs. This is precisely the coupling that the refactor should remove.

Target interpretation after refactor.

$$\texttt{Node} = \texttt{Vertex} + \texttt{field attachment}.$$

This does not necessarily mean inheritance. The important point is semantic: `Node` belongs to the analysis/model layer, not to the geometry/domain layer.

11.2.4 `IntegrationPoint<dim>`

`IntegrationPoint` is a sampling site used by numerical integration. It stores physical coordinates, a quadrature weight, and an identifier.

What it is. A numerical quadrature sample attached to an element geometry.

What it is not. It is not a mesh vertex and not a constitutive model by itself.

Important nuance. An integration point represents a *role*, not merely a location. Two different entities can occupy the same coordinates and still not be the same thing. This is crucial for understanding Lobatto collocation and mixed formulations.

Potential improvement. Eventually, `IntegrationPoint` should probably be understood as a *sample site* with:

- a physical coordinate;
- a reference coordinate;
- a numerical weight;
- possibly the differential measure $|J|$ or a cached metric.

The current implementation already covers the first and third items.

11.2.5 `MaterialPoint<MaterialPolicy>`

`MaterialPoint` is the constitutive analogue of a continuum Gauss point. It binds a `Material<Policy>` to an `IntegrationPoint`.

What it is. A constitutive state machine attached to a continuum sampling site.

What it is not. It is not geometry, not a mesh node, and not a quadrature rule.

Mathematical meaning. For a continuum element, the geometric interpolation produces the kinematic state at a sampling location; the material point then stores and updates the constitutive history at that location:

$$\texttt{ElementGeometry} \longrightarrow \texttt{IntegrationPoint} \longrightarrow \texttt{MaterialPoint}.$$

11.2.6 `SectionGeometry`

`SectionGeometry` is not a point at all. It is the geometric description of a structural cross-section: area, moments of inertia, torsional constant, and shear factors.

What it is. Cross-section *shape* data.

What it is not. It is not a location, not a node, and not a constitutive point.

Architectural consequence. The name is correct, but it must stay clearly separated from both `PetscSection` and `MaterialSection`.

11.2.7 `NodeSection<Dim, Geom>`

`NodeSection` combines:

1. a nodal location;
2. a local section frame;
3. a cross-section geometry.

What it is. A *section station* located at a node, that is, a nodal section plane with shape data and orientation.

What it is not. It is not a constitutive sampling point unless a formulation explicitly chooses to collocate section constitutive evaluation at that nodal station.

Why this distinction matters. In many beam and shell formulations, the nodal section plane is a geometric interpolation entity, whereas the constitutive update happens at different quadrature stations.

11.2.8 `MaterialSection<MaterialPolicy>`

`MaterialSection` is the structural analogue of `MaterialPoint`. It binds a section constitutive law to an integration point along a beam axis or shell midsurface.

What it is. A constitutive state machine attached to a structural sampling site.

What it is not. It is not the same as `SectionGeometry`, and not the same as `NodeSection`.

Mathematical meaning. It stores and updates generalized stress-resultants as functions of generalized strain measures:

$$\texttt{generalized strains} \mapsto \texttt{section resultants}.$$

Architectural observation. `MaterialPoint` and `MaterialSection` are role siblings. The first is a continuum constitutive site; the second is a structural constitutive site.

11.2.9 `PetscSection`

Although it is not a C++ entity defined in this library, it deserves explicit mention because it introduces the strongest terminological collision.

What it is. A PETSc map from mesh points to numbers of DOFs, offsets, and constraints.

What it is not. It is neither a structural section nor a constitutive section.

Architectural rule. `PetscSection` belongs to the solver/indexing infrastructure and should never be allowed to contaminate the semantics of structural section objects.

11.3 Coincidence Versus Identity

The most important conceptual rule for the refactor is the following:

> Two entities may coincide in space without being the same numerical object.

This rule resolves the apparent paradoxes raised by Lobatto quadrature and mixed formulations.

11.3.1 Case 1: Gauss–Lobatto beam quadrature

Suppose a beam formulation uses Gauss–Lobatto integration. Then the end quadrature points lie exactly at the beam endpoints. If the beam also uses nodal section stations at the endpoints, then:

- the *nodal section station* at the endpoint;
- the *quadrature point* at the endpoint;
- the *constitutive section sample* at the endpoint

all share the same physical coordinates.

That does *not* mean they should necessarily be represented by the same object. Their roles are distinct:

- the nodal station belongs to interpolation and field attachment;
- the quadrature point belongs to integration;
- the constitutive section sample belongs to material evolution.

The right design principle is therefore:

> collocation should be represented as a relationship, not as ontological collapse.

In practice, this means that a Lobatto endpoint can be modelled as *three roles sharing one location*, rather than as a single overloaded object.

11.3.2 Case 2: mixed or assumed-strain formulations

In formulations such as degenerated Timoshenko beams or MITC shell families, some sampling sites may acquire independent unknowns, tying variables, or assumed-strain parameters.

Here again, the key is to separate:

1. the geometric location of the sample;
2. the fact that it is used in integration;
3. the fact that additional discrete unknowns live there.

An integration point with independent unknowns is therefore *not* "a node in disguise"; rather, it is a sampling site that additionally plays the role of a field carrier.

Recommended interpretation. The library should treat this as:

$$\texttt{SampleSite} + \texttt{FieldAttachment},$$

not as a reason to redefine `IntegrationPoint` itself as a node.

This preserves both conceptual clarity and extensibility. The same sampling site can later carry:

- a constitutive law only;
- constitutive law plus internal variables;
- constitutive law plus independent element-local DOFs;
- tying or assumed-strain data without constitutive history.

11.4 Proposed Ontology for the Refactor

The cleanest layered ontology for `fall_n` is the following.

11.4.1 Layer 0: pure geometry/topology

- `Point<Dim>`: coordinates only.
- `Vertex<Dim>`: point + mesh identity.

- `ElementGeometry<Dim>`: mapped geometry, reference interpolation, quadrature mapping, face extraction.
- `Domain<Dim>`: owns vertices, element geometries, boundary groups, and PETSc mesh materialisation support, but no DOFs and no constitutive state.

11.4.2 Layer 1: discrete field attachment

- `Node`: a field-bearing attachment to a vertex, carrying nodal DOFs and constraints.
- future internal field sites: element-local unknown carriers for mixed methods or assumed-strain formulations.
- `PetscSection`: global indexing metadata for those field sites.

11.4.3 Layer 2: sampling and constitutive sites

- `IntegrationPoint`: numerical sample site.
- `MaterialPoint`: continuum constitutive site.
- `MaterialSection`: structural constitutive site.
- future tying or assumed-strain sites: sampled kinematic fields that may or may not have constitutive state.

11.4.4 Layer 3: structural section semantics

- `SectionGeometry`: cross-section shape properties.
- `NodeSection`: nodal section station = node + frame + section geometry.
- section constitutive relations (`TimoshenkoBeamSection3D`, `MindlinShellSection`, `FiberSection`): map generalized strains to resultants.

This layering produces a clean separation between:

1. *where something is*;
2. *how it is sampled*;
3. *what unknowns live there*;
4. *what constitutive law is evaluated there.*

11.5 Implications for `Domain`

Under this ontology, `Domain` should evolve toward the following responsibilities:

1. ownership of mesh vertices;
2. ownership of element geometry objects;
3. storage of boundary groups and physical labels;
4. translation to `DMPlex`;
5. topology and geometry queries.

And it should explicitly *not* own:

1. nodal DOF layouts;
2. constitutive points;
3. material state;
4. structural section stations;
5. solver vectors or matrices.

This has a direct consequence for the current code:

> `Domain` owning `Node` is architecturally temporary, not final.

The first code step taken in this iteration is therefore the introduction of `Vertex<Dim>` as the future geometry-level entity.

11.6 HPC and Compile-Time Guidance

The user requirement for the refactor is not only architectural cleanliness but also efficiency in a high-performance context. That requirement strongly favours a few design rules.

11.6.1 Prefer static structure in hot paths

The current library already uses static polymorphism effectively in the element kernel. That should continue. In particular:

- shape functions, quadrature rules, and reference-cell topology should remain compile-time objects whenever possible;
- constitutive updates should stay inside element-local loops with no new virtual dispatch;
- small fixed-size Eigen matrices are still the right choice for local operators.

11.6.2 Keep `Domain` simple, not hyper-generic

The right way to favour HPC here is *not* to template `Domain` on every imaginable backend or policy. That would increase compile times, fragment code paths, and create speculative abstraction.

The efficient choice is:

- keep `Domain` concrete and PETSc-oriented;
- separate concepts cleanly;
- minimise includes and cross-layer dependencies;
- keep backend-specific code concentrated in a few classes.

11.6.3 Avoid type collapse when roles overlap

Role overlap should be represented through composition and index maps rather than inheritance-heavy unions of responsibilities.

This means:

- a Lobatto endpoint may be both nodal station and quadrature sample, but that is best represented by shared coordinates or shared IDs, not by redefining one type to become the other;
- a mixed formulation with internal DOFs at sample sites should introduce a dedicated field attachment for those sites, not redefine every integration point in the library as a node.

11.6.4 Recommended storage philosophy

For the next stages of the refactor, the following storage strategy is the most coherent with the current codebase:

- `Vertex`: compact aggregate, trivially copyable if possible;
- `Node`: inline or SBO DOF storage as already implemented;
- `IntegrationPoint`: compact POD-like sample record;
- constitutive points: element-local vectors, because they are naturally owned by elements and evolved elementwise;
- PETSc indexing: stored in model/solver-side data structures, not in the geometric domain.

This preserves cache locality while keeping compile-time structure strong and runtime dispatch localized.

11.7 Migration Plan

The refactor should proceed incrementally.

Phase 1: fix ontology

- introduce `Vertex`;
- document the role separation;
- stop using "section" ambiguously in new code.

Phase 2: decouple geometry from nodal unknowns

- move `Domain` toward `std::vector<Vertex<dim>>`;
- make `ElementGeometry` depend on a geometry-only vertex concept rather than directly on `Node`;
- relocate DOF assignment and constraints fully to `Model`.

Phase 3: make field attachments explicit

- define how primary nodal unknowns are attached to vertices;
- introduce dedicated internal field sites for mixed methods when needed;
- keep collocation and coincidence as relationships, not type collapse.

Phase 4: unify constitutive-site patterns

- consider a shared abstraction behind `MaterialPoint` and `MaterialSection`;
- preserve policy-based constitutive dispatch and element-local storage.

11.8 Conclusion

The central conclusion of this chapter is simple:

> The library does not suffer from having too many abstractions; it suffers from having a few abstractions that currently play more than one role.

The remedy is therefore not more runtime polymorphism, but better ontological separation:

- `Point` for coordinates;
- `Vertex` for mesh geometry/topology;
- `Node` for field attachment and DOFs;
- `IntegrationPoint` for numerical sampling;
- `MaterialPoint`/`MaterialSection` for constitutive state;
- `SectionGeometry`/`NodeSection` for structural section geometry and nodal section stations.

Once this distinction is enforced, the `Domain` refactor becomes much less ambiguous. The domain can stay geometrically clean, the model can own the discrete unknowns, and the element layer can continue to exploit strong compile-time structure without sacrificing conceptual clarity.

Chapter 12

Domain Vertex Transition

Chapter 11 fixed the ontology of the refactor. This chapter documents the next implementation step: the domain now treats `Vertex` as the canonical geometric storage, while `Node` survives as a transitional analysis-layer cache.

The design goal of this step is deliberately modest:

> move the ownership boundary of `Domain` toward pure geometry without breaking the current element, model, and PETSc assembly pipelines.

That means this is not yet the final state of the refactor. It is the bridge state that keeps existing kernels working while removing the strongest geometric/analysis conflation.

12.1 Motivation

Before this step, `Domain` owned a vector of `Node<dim>`. That had three consequences:

1. the geometric domain depended directly on nodal DOF storage;
2. simply constructing a domain implied that the domain was already in an analysis-ready state;
3. purely geometric consumers such as mesh export were forced to read geometry through analysis nodes.

This was workable, but it obscured the intended layering:

$$\texttt{Domain} \not\equiv \texttt{Model}.$$

The refactor therefore changes the ownership model to:

`Domain owns vertices,  Model / assembly consume nodes.`

12.2 Implemented Transition Design

12.2.1 Canonical storage

`Domain` now stores

`std::vector<Vertex<dim>>`

as its primary geometric container. This is the authoritative representation of mesh points for:

- coordinates;
- mesh identity;
- sorting and lookup by vertex ID;
- geometry-only consumers.

12.2.2 Node cache

To preserve compatibility with the current element and model infrastructure, `Domain` also materialises a

```
std::vector<Node<dim>>
```

cache on demand. This cache is derived from the vertices and is used only when analysis-level operations require nodal DOFs or PETSc indexing.

This means the runtime interpretation is now:

- `vertices_` is the geometric truth;
- `nodes_cache_` is a derived analysis view.

Why this is acceptable. The cache is not introduced for performance in the hot path. It is a compatibility layer during migration. Since nodes are created once and then reused during analysis, the cost is negligible relative to element assembly and material updates.

12.2.3 Cache invalidation rule

Whenever the vertex container changes, the node cache is invalidated. The next request for `nodes()`, `node(id)`, or element-node binding rebuilds the cache.

This makes the design explicit:

> geometry changes invalidate analysis views.

That is the correct dependency direction.

12.3 Why This Preserves Performance

The user requirement for this refactor was clear: do not sacrifice runtime performance unnecessarily, and favour compile-time structure suitable for HPC.

This implementation respects that requirement in three ways.

12.3.1 No change to element hot loops

The integration, constitutive update, and local stiffness computations remain exactly where they already were:

- `ElementGeometry`;
- `ContinuumElement`;
- `BeamElement`;
- `TimoshenkoBeamN`;
- `ShellElement`.

Those kernels still operate on bound `Node` objects and continue to benefit from fixed-size matrices, compile-time quadrature, and policy-based dispatch.

12.3.2 Geometry-only consumers stop depending on nodes

The VTK mesh-export path was updated to read coordinates from `Domain::vertices()` rather than from `Domain::nodes()`. This is architecturally cleaner and slightly cheaper conceptually: geometry is now consumed as geometry.

12.3.3 PETSc creation is now lazy

An important side effect of the refactor is the change to `Mesh`. The `DM` object is no longer created in the `Mesh` constructor. Instead, it is created lazily when the DMPlex-specific operations are first needed.

This has two benefits:

1. a purely geometric `Domain` can be created and manipulated without immediately entering PETSc runtime semantics;
2. tests that exercise only the geometric/domain side no longer require `PetscInitialize`.

Architecturally, this is important because it moves PETSc from "always present" to "activated when the backend is actually used".

12.4 Code-Level Changes

The main code changes in this phase are:

1. `Vertex<Dim>` introduced earlier becomes the canonical storage type inside `Domain`.
2. `Domain` gains explicit vertex accessors: `vertices()`, `vertex(id)`, `num_vertices()`.
3. `Domain::nodes()` becomes a lazily materialised analysis view.
4. `GmshDomainBuilder` now sorts the domain through `sort_vertices_by_id()` rather than sorting the nodal view.
5. `VTKModelExporter` and the legacy `VTKDataContainer` now use vertices for mesh coordinates.
6. `Mesh` now creates the PETSc `DM` lazily.

This is intentionally a minimal-surface-area change. The aim was to shift the ownership boundary without forcing an immediate rewrite of `ElementGeometry` or `Model`.

12.5 What Remains Transitional

This phase does *not* yet complete the domain refactor.

The following are still transitional:

- `ElementGeometry` still binds concrete `Node` pointers, not geometry-only vertex handles.
- `Model` still reads and writes nodal DOF data through the domain-level node view.
- PETSc sieve IDs are still stored in `Node`, not yet in a dedicated analysis/indexing layer.

That is acceptable at this stage. The important structural improvement is that these analysis concerns are no longer the *canonical ownership model* of the domain itself.

12.6 Validation

This phase adds a dedicated test executable for the transition.

12.6.1 Domain/Vertex refactor test

The new test checks:

1. that vertices are the canonical stored geometry;
2. that the nodal view is materialised from those vertices;
3. that sorting vertices updates the nodal view coherently;
4. that analysis-only data written into the node cache does not alter the geometric meaning of the corresponding vertex;
5. that the legacy `add_node(Node&&)` path still preserves geometry correctly.

12.6.2 Regression coverage

The existing serendipity test suite was recompiled and rerun after this change, which is a useful regression check because it exercises:

- `ElementGeometry`;
- quadrature;
- VTK export;
- geometry binding through nodes.

In addition, the Gmsh pipeline target was rebuilt successfully, confirming that the builder and domain interface remain consistent at compile time.

12.7 Architectural Consequences

This transition has one especially important architectural consequence:

> `Domain` can now be understood as a geometric object that happens to be able to provide an analysis view, rather than as an analysis object that also stores geometry.

That inversion is subtle, but it matters. It clarifies the dependency direction for the next phase:

1. first, geometry is owned by the domain;
2. second, analysis nodes are attached or materialised from it;
3. third, elements and solvers consume those analysis nodes.

This is much closer to the ontology laid out in the previous chapter.

12.8 Next Step

The next logical step is now well defined:

> remove the direct `Node`-dependence from `ElementGeometry`.

That will require introducing a geometry-facing vertex handle or concept for binding coordinates, while the model layer separately manages nodal DOFs and PETSc indexing.

Once that is done, `Domain` will no longer need to expose a nodal cache at all, except perhaps as a legacy adapter.

12.9 Conclusion

This chapter marks the first real implementation step of the domain refactor. The main result is not a new feature but a new ownership boundary:

- vertices own domain geometry;
- nodes are an analysis view;
- mesh export reads geometry from vertices;
- PETSc mesh backend creation is deferred until needed.

No element hot path was slowed down, no quadrature logic was generalized away, and existing element kernels continue to operate unchanged. That is the right kind of progress for this refactor: architectural movement without numerical regression.

Chapter 13

Geometry-Only Element Binding

13.1 Context

After the first `Domain` transition, the library already stored `Vertex<Dim>` as the canonical geometric entity while keeping a lazy `Node<Dim>` cache for analysis. That solved the ownership problem in `Domain`, but a structural coupling remained in the element layer: `ElementGeometry` and the concrete element families still used `Node` as the source of physical coordinates.

Architecturally, that was still inconsistent with the ontology established in Chapter 11. A `Node` is an analysis attachment that carries degrees of freedom, PETSc numbering, and constraints. Geometry, however, only needs physical coordinates. The second phase of the refactor therefore separates these two bindings inside the element geometry layer.

13.2 Problem Statement

Before this change, a tensor-product, simplex, or serendipity element stored a single pointer array

$$\{\texttt{Node<dim>*}\}_{a=1}^{n_{\text{node}}}$$

and used that same array both for:

1. geometric interpolation and Jacobian evaluation, and
2. analysis-level access to nodal degrees of freedom.

This conflated two different roles:

$$\text{geometry site} \neq \text{analysis node}.$$

Even when both roles are collocated in space, their semantics are distinct. A continuum element can be fully geometrically defined from vertices without any DOF allocation. Conversely, an analysis formulation may decorate a point with DOFs, constraints, or solver indices without changing its geometric role.

13.3 Design Objective

The goal of this phase was to make the geometry layer depend only on the minimum abstraction required for isoparametric mapping:

`Point<Dim>` or a type derived from it.

At the same time, the analysis layer still needs direct access to `Node<Dim>` once PETSc sections and elemental DOFs are assigned.

The desired split is therefore:

`ElementGeometry` $\longrightarrow$ `Point/Vertex binding`	for $x(\boldsymbol{\xi}), J(\boldsymbol{\xi}), \mathrm{d}\Omega$,
`ElementGeometry` $\longrightarrow$ `Node binding`	for DoFs, constraints, and PETSc layout.

13.4 Implemented Refactor

13.4.1 New dual binding inside concrete elements

The three concrete element families

- `LagrangeElement`,
- `SimplexElement`, and
- `SerendipityElement`

now hold two logically distinct pointer arrays:

```
points_p : Point<Dim>*
nodes_p : Node<Dim>*
```

The geometry path uses only `points_p`. In particular:

- `coord_array()`,
- `map_local_point()`, and
- `evaluate_jacobian()`

no longer depend on `Node`.

The analysis path continues to use `nodes_p`. Methods such as `node_p()` and all element-level DOF collection code remain unchanged.

13.4.2 Wrapper-level support in `ElementGeometry`

The type-erased wrapper was extended with:

- `bind_point(i, Point<dim>*)`,
- `point_p(i)`,
- existing `bind_node(i, Node<dim>*)`, and
- existing `node_p(i)`.

This preserves the previous public API for analysis code while adding an explicit geometry-only binding channel.

13.4.3 Key compatibility rule

Whenever a `Node` is bound, the corresponding geometry point is also bound automatically. In symbols:

$$\texttt{bind_node}(i, n) \Longrightarrow \texttt{bind_point}(i, n).$$

This means:

- legacy code that only binds nodes still works unchanged;
- new geometry-first code can bind vertices before any DOFs exist.

13.5 Domain-Level Consequences

13.5.1 Geometry-first linking

`Domain` now exposes a dedicated `link_geometry_to_elements()` phase. It binds each element node id to its canonical `Vertex<dim>`:

$$\texttt{element.node(i)} \mapsto \texttt{vertex_by_id[element.node(i)]}.$$

This enables:

- geometric mapping,
- Jacobian evaluation,
- integration point placement, and
- geometry-driven boundary extraction

without first materialising the analysis-node cache.

13.5.2 Boundary extraction no longer needs `node_p().id()`

The plane-based boundary builder previously queried geometric membership through `elem.node_p(local).id(` which forced the existence of node bindings. After the refactor, face detection uses only the canonical element connectivity:

`elem.node(local).`

The resulting boundary element is then:

1. created from the face connectivity,
2. immediately bound to `Vertex` objects for geometry, and
3. optionally bound to `Node` objects if the analysis cache already exists.

13.5.3 Sieve assembly remains analysis-oriented

The PETSc sieve assembly still needs analysis nodes because sieve ids and DOF layout belong to the solver layer. However, even there the distinction is now cleaner:

- geometry binding prepares the element for integration and mapping;
- node binding prepares the element for DOF and PETSc interaction.

13.6 Why this is efficient

13.6.1 No new runtime polymorphism in the hot path

The critical loops for element integration remain unchanged:

- quadrature rule selection is still static via templates,
- local reference-space evaluation remains compile-time typed,
- Jacobian and interpolation work on fixed-size Eigen objects,
- the type-erased wrapper is still crossed only at the same boundary as before.

The only additional state is a pointer array to `Point<Dim>` inside each concrete element. Accessing `coord(i)` through a `Point` base reference is not virtual; it is just a normal base-subobject access. Therefore this refactor does not introduce a new dynamic-dispatch cost in the Gauss loop.

13.6.2 Compile-time impact

The change slightly increases template surface area in the element headers, but the trade-off is favourable:

- `Domain.hh` no longer needs to include Gmsh reader details;
- geometry-only workflows can avoid dragging analysis semantics into lower layers;
- future specialisations can target `Point`/`Vertex` independently of PETSc-related analysis infrastructure.

For an HPC-oriented codebase this is important because the hottest kernels remain statically shaped, while architectural boundaries become cleaner and easier to optimise separately.

13.7 Validation

The test suite was extended with geometry-first checks in the `domain_vertex_refactor` executable:

1. a quadrilateral element can map its centre and integrate area after binding only vertices;
2. a boundary element generated by `create_boundary_from_plane()` can map and integrate from vertex binding alone.

These tests validate the new invariant:

element geometry is usable as soon as geometry is bound, even if analysis nodes have not yet been materialised.

13.8 Architectural Interpretation

This phase does not eliminate `Node` from the library. It puts `Node` back in its proper role.

- `Vertex`: canonical domain geometry/topology entity;
- `Point`: minimal coordinate carrier;
- `ElementGeometry`: geometric interpolation operator over bound points;
- `Node`: analysis decorator attached later when the model introduces DOFs.

That separation is especially important for future mixed and structural formulations, where collocation between geometry sites, integration sites, and DOF-carrying sites may occur but should not collapse the underlying concepts.

13.9 Next Step

The next logical phase is to propagate this distinction upward:

- reduce remaining read paths that only need connectivity ids but still access `node_p()`;
- make geometry-only exporters and preprocessors depend only on `Vertex`/`Point`;
- move more of the solver-specific state management out of `Domain` and into the model/-analysis side.

At that stage, `Domain` will be geometrically pure not only in storage, but also in the majority of its operational paths.

Bibliography

[1] Satish Balay, Shrirang Abhyankar, Mark F. Adams, et al. PETSc Users Manual (revision 3.24). `https://petsc.org/release/docs/manual/`, 2024. Argonne National Laboratory, ANL-21/39.

[2] Philippe G. Ciarlet and Pierre-Arnaud Raviart. General Lagrange and Hermite interpolation in $\mathbb{R}^n$ with applications to finite element methods. *Archive for Rational Mechanics and Analysis*, 46(3):177–199, 1972.

[3] Pooyan Dadvand, Riccardo Rossi, and Eugenio Oñate. An object-oriented environment for developing finite element codes for multi-disciplinary applications. *Archives of Computational Methods in Engineering*, 17(3):253–297, 2010.

[4] Michael G. Duffy. Quadrature over a pyramid or cube of integrands with a singularity at a vertex. *SIAM Journal on Numerical Analysis*, 19(6):1260–1262, 1982.

[5] Walter Gautschi. *Orthogonal Polynomials: Computation and Approximation.* Numerical Mathematics and Scientific Computation. Oxford University Press, 2004.

[6] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009.

[7] Gene H. Golub and John H. Welsch. Calculation of Gauss quadrature rules. *Mathematics of Computation*, 23(106):221–230, 1969.

[8] A. Grundmann and H. M. Möller. Invariant integration formulas for the n-simplex by combinatorial methods. *SIAM Journal on Numerical Analysis*, 15(2):282–290, 1978.

[9] Preston C. Hammer, O. J. Marlowe, and Arthur H. Stroud. Numerical integration over simplexes and cones. *Mathematical Tables and Other Aids to Computation*, 10(55):130–137, 1956.

[10] Patrick Keast. Moderate-degree tetrahedral quadrature formulas. *Computer Methods in Applied Mechanics and Engineering*, 55(3):339–348, 1986.

[11] Frank McKenna, Gregory L. Fenves, Michael H. Scott, and Boris Jeremić. Open system for earthquake engineering simulation (OpenSees). *Pacific Earthquake Engineering Research Center, University of California, Berkeley*, 2000. `https://opensees.berkeley.edu/`.

[12] Arthur H. Stroud. *Approximate Calculation of Multiple Integrals.* Prentice-Hall Series in Automatic Computation. Prentice-Hall, 1971.

[13] Gábor Szegő. *Orthogonal Polynomials.* Colloquium Publications. American Mathematical Society, 4th edition, 1975.

[14] O. C. Zienkiewicz and J. Z. Zhu. The superconvergent patch recovery and a posteriori error estimates. Part 1: The recovery technique. *International Journal for Numerical Methods in Engineering*, 33(7):1331–1364, 1992.

[15] O. C. Zienkiewicz and J. Z. Zhu. The superconvergent patch recovery and a posteriori error estimates. Part 2: Error estimates and adaptivity. *International Journal for Numerical Methods in Engineering*, 33(7):1365–1382, 1992.